Herramienta de simulación para dar soporte a la enseñanza de arquitectura de computadoras

Maestría en Sistemas de Información

Ruiz Jose Maria

Director: Colombani Marcelo Alberto

2024

Índice

Resumen **1**

Agradecimientos **2**

1 Introducción **3**
1.1 Justificación 5
1.2 Objetivos 6
1.3 Metodología de desarrollo 6
1.4 Organización del documento 7

2 Estado del arte **8**
2.1 Arquitectura de computadoras 8
2.2 Arquitecturas Von Neumann y Harvard 9
2.3 Tipos de arquitecturas 11
2.4 Repertorio de instrucciones 13
2.5 Filosofías CISC y RISC 19
2.6 Arquitectura x86 23
2.7 Lenguaje ensamblador 27
2.8 Simulación 28
2.9 Simulación en la educación 29
2.10 El Formalismo DEVS (Discrete Event System Specification) 31

3 Comparativa de simuladores **32**
3.1 Estudios similares 32
3.2 Simuladores bajo análisis 33
3.3 Criterios de evaluación 33
3.4 Selección de simuladores 35
3.5 Participantes en la evaluación 35
3.6 Análisis comparativo 36
3.7 Resultados 37

4 Diseño y Construcción del Simulador **40**
4.1 Requisitos de la Herramienta 40
4.2 Arquitectura propuesta 41
4.3 Pruebas 50

4.4 Portabilidad 51
4.5 Mantenibilidad 51
4.6 Escalabilidad 51

Bibliografía **52**

Índice de cuadros

2.1 Comparación de Arquitecturas 13

2.2 Modos de direccionamiento básicos 17

2.3 Comparativa entre CISC y RISC 22

2.4 Hitos procesadores x86 23

2.5 Línea de Tiempo de la Evolución de la Arquitectura x86 24

2.6 Comparativa de Ensambladores x86 28

3.1 Comparativa según criterios de evaluación preestablecidos 37

4.1 Arquitectura x86 propuesta 42

4.2 Tabla de Instrucciones y Códigos de Operación de la Arquitectura x86 43

4.3 Tabla de codififación de instrucciones 44

4.12 Tabla de Instrucciones y Códigos de Operación de la Arquitectura x86 49

Índice de figuras

2.1 Arquitectura Von Neumann 10

2.2 Arquitectura Harvard 11

2.3 Características repertorio de instrucciones 14

2.4 Modos de direccionamiento 16

2.5 Formato de instrucciones 19

2.6 Formato de instrucciones del Pentium x86 26

Resumen

Existe un consenso creciente en el uso de herramientas de simulación en la enseñanza para procesos dinámicos complejos, como las operaciones intrínsecas de la computadora, que permiten representar de forma visual e interactiva la organización y arquitectura interna de la computadora, facilitando así la comprensión de su funcionamiento por parte de los alumnos y el desarrollo de los temas por parte del docente. En este contexto, los simuladores juegan una pieza clave en el campo de la Arquitectura de Computadoras, permitiendo conectar fundamentos teóricos con la experiencia práctica, simplificando abstracciones y haciendo más rica la labor docente. La arquitectura x86 es ampliamente utilizada en computadoras de escritorio y servidores. Este documento pretende realizar una comparación de los simuladores x86 que más se adecuan en el dictado de la asignatura Arquitectura de Computadoras de la carrera Licenciatura en Sistemas, establecer los criterios de evaluación y evaluar los simuladores seleccionados de acuerdo con estos criterios.

La presente investigación surge en el marco del proyecto de investigación I/D novel PID-UNER 7065: "Enseñanza/aprendizaje de asignatura Arquitectura de Computadoras con herramientas de simulación de sistemas de cómputos". El Proyecto es llevado a cabo en la Facultad de Ciencias de la Administración de la Universidad Nacional de Entre Ríos, se vincula directamente con la asignatura Arquitectura en Computadoras que se dicta en segundo año de la carrera Licenciatura en Sistemas perteneciente a la Facultad de Ciencias de la Administración de la Universidad Nacional de Entre Ríos.

Palabras clave: x86, simulador, aprendizaje, enseñanza, arquitectura de computadoras.

Agradecimientos

Agradecimientos aquí.

Capítulo 1

Introducción

En nuestra vida cotidiana, utilizamos dispositivos como computadoras de escritorio, teléfonos y relojes inteligentes, todos ellos basados en arquitecturas específicas. Comprender su funcionamiento e interacciones nos permite diseñar y desarrollar aplicaciones más eficientes.

Es esencial que los estudiantes de Arquitectura de Computadoras comprendan tanto la estructura como el funcionamiento interno de una computadora, y adquieran experiencia práctica con ellas. Para lograrlo, es fundamental disponer de un laboratorio bien equipado con el hardware adecuado y suficiente tiempo para que los estudiantes se familiaricen con las herramientas prácticas. En este contexto, se han desarrollado numerosos simuladores que facilitan la comprensión de la estructura y el funcionamiento de las computadoras, proporcionando valiosas experiencias de aprendizaje.

Esta tesis, inscrita en la Maestría en Sistemas de Información de la Facultad de Ciencias de la Administración, está directamente vinculada con el proyecto de investigación I/D novel PID-UNER 7065, titulado "Enseñanza/aprendizaje de Arquitectura de Computadoras con herramientas de simulación de sistemas de cómputo", desarrollado en la Facultad de Ciencias de la Administración de la Universidad Nacional de Entre Ríos [1].

La asignatura Arquitectura de Computadoras forma parte del plan de estudios de la carrera de Licenciatura en Sistemas, Universidad Nacional de Entre Ríos. Su objetivo es que los estudiantes comprendan la estructura y funcionamiento de las computadoras, y la ejecución lógica de un programa a nivel de instrucciones de máquina.

El primer paso para comprender las computadoras es entender que son máquinas que toman datos del exterior, los procesan y almacenan los resultados en la memoria o los envían a dispositivos de entrada y salida.

El procesamiento se realiza a través del procesador o CPU, y es en este componente donde los estudiantes encuentran mayor complejidad y dificultades para comprender su funcionamiento.

A pesar de que es posible explicar las partes del procesador, su funcionamiento, la interacción de sus componentes y enseñar lenguaje ensamblador mediante prácticas, los estudiantes suelen tener dificultades para lograr una comprensión completa del funcionamiento.

Sin embargo, la utilización de simuladores permite afianzar los conocimientos de los temas vistos en las clases teóricas, a fin de evitar que los estudiantes desvíen su atención hacia el aprendizaje del simulador propiamente dicho, es necesario que estos sean de manejo simple, intuitivo y visualmente atractivo, simplificando su aprendizaje de su uso.

La simulación es un término de uso diario en muchos contextos: medicina, militar, entretenimiento, educación, etc., debido a que permite ayudar a comprender cómo funciona un sistema, responder preguntas como "qué pasaría si", con el fin de brindar hipótesis sobre cómo o por qué ocurren ciertos fenómenos.

Para continuar, se define simulación como el proceso de imitar el funcionamiento de un sistema a medida que avanza en el tiempo. Entonces para llevar a cabo una simulación, es necesario desarrollar previamente un modelo conceptual que representa las características o comportamientos del sistema, mientras que la simulación representa la evolución del modelo a medida que avanza en el tiempo [2][3][4].

Con los avances en el mundo digital, la simulación se ha convertido en una metodología de solución de problemas indispensable para ingenieros, docentes, diseñadores y gerentes. La complejidad intrínseca de los sistemas informáticos los hace difícil comprender y costosos de desarrollar sin utilizar simulación [3].

Muchas veces en el ámbito educativo, resulta difícil transmitir fundamentos teóricos de la organización y arquitectura interna de las computadoras debido a la complejidad de los procesos involucrados. Si sólo incorporamos los medios de enseñanza tradicionales, como ser una pizarra, un libro de texto o diapositivas, los mismos tienen una capacidad limitada para representar estos fundamentos. En consecuencia, es imprescindible un alto nivel de abstracción por parte del estudiante para desarrollar un modelo mental adecuado para capturar la organización y arquitectura interna de las computadoras [5][6][7].

Es evidente la necesidad de utilizar nuevas tecnologías como recursos didácticos y medios de transferencia de conocimiento, ya que ayudan a los estudiantes a relacionar conceptos abstractos con realidades concretas. Estas tecnologías permiten situar al estudiante en un contexto que imita aspectos de la realidad, facilitando la detección de problemáticas similares a las que podrían ocurrir en situaciones reales. Este enfoque promueve un mejor entendimiento a través del trabajo exploratorio, la inferencia, el aprendizaje por descubrimiento y el desarrollo de habilidades [8][9].

Un simulador de arquitectura es una herramienta que imita el hardware de un sistema, representando sus aspectos arquitectónicos y funciones. Permiten realizar cambios,

pruebas y ejecutar programas sin riesgo de dañar componentes ni depender de equipos físicos disponibles [10].

Algunas herramientas ofrecen una representación en forma visual e interactiva de la organización y arquitectura interna de la computadora, facilitando así la comprensión de su funcionamiento. En este sentido, los simuladores juegan una pieza clave en el campo de la Arquitectura de Computadores, permitiendo conectar fundamentos teóricos con la experiencia práctica, simplificando abstracciones y facilitando la labor docente [11][12][13][14].

El repertorio de instrucciones de la arquitectura x86, ampliamente utilizado en computadoras de escritorio y servidores, comenzó con el procesador Intel 8086 en 1978 como una arquitectura de 16 bits. Evolucionó a una arquitectura de 32 bits con el procesador Intel 80386 en 1985 (i386 o x86-32) y posteriormente a 64 bits con las extensiones de AMD (AMD64) y su adopción por Intel (Intel 64) [15][16].

Un procesador x86-64 mantiene la compatibilidad con los modos x86 existentes de 16 y 32 bits, y permite ejecutar aplicaciones de 16 y 32 bits, como así también de 64 bits. Esta compatibilidad hacia atrás protege las principales inversiones en aplicaciones y sistemas operativos desarrollados para la arquitectura x86 [15][16][17].

Por ello, la enseñanza de la arquitectura x86 es de gran relevancia en la asignatura Arquitecturas de Computadoras debido a los diferentes temas que aborda.

Para brindar esta experiencia, es necesario un laboratorio equipado con el hardware adecuado y tiempo suficiente para que los estudiantes se familiaricen con las herramientas. Por este motivo, se han desarrollado muchos simuladores que facilitan la comprensión del funcionamiento y la estructura del computador, ofreciendo valiosas experiencias de aprendizaje [18].

1.1 Justificación

Aunque ya existen simuladores de la arquitectura x86 que apoyan la enseñanza en los cursos de Arquitectura de Computadoras [10][11], estos suelen presentar una gran cantidad de contenidos preestablecidos. Si bien estos contenidos son relevantes, ofrecer toda la especificación de la arquitectura x86 desde el principio puede ser abrumador para los estudiantes y dificultar su comprensión. Sin embargo, desde esta tesis se propone un enfoque diferente: desarrollar una herramienta de simulación de la arquitectura x86 para apoyar la enseñanza de los contenidos específicos de la currícula de Arquitectura de Computadoras. Partiendo de una visión global de la estructura y funcionamiento de la computadora (CPU, memoria, módulo de E/S y buses), mostrando los micropasos necesarios para la realización del ciclo básico de una instrucción, ofreciendo un repertorio reducido de instrucciones que se habiliten las instrucciones a medida que se dictan en la asignatura, permitiendo la generación y ejecución de

programas escritos en lenguaje ensamblador, ya sea paso a paso por instrucción o completa, gestión básica de interrupciones permitiendo la interacción con el teclado y la pantalla, comunicación con los módulos de entrada y salida e interacciones con los periféricos, y por último, medidas de rendimiento sobre la ejecución de un programa.

Con el objeto de ofrecer al estudiante un simulador bien diseñado, robusto, modular y por tanto flexible y sencillo de modificar o ampliar, se explorará la utilización de técnicas formales de modelización y simulación como las redes de Petri o DEVS (Discrete Event System Specification). Estas técnicas permiten separar conceptualmente las capas de modelización y simulación y ofrecen por ello una separación ortogonal de ambas, facilitando la comprensión y modificación del software. Además, permiten el escalado transparente de las simulaciones, pudiéndose ejecutar en entornos de cómputo paralelo o distribuido sin necesidad de modificar el modelo en sí, lo que aporta grandes ventajas de escalado [19][20][21].

1.2 Objetivos

El objetivo principal de esta tesis es construir una herramienta de simulación de la arquitectura x86 para apoyar la enseñanza de arquitectura de computadoras, enfocándose específicamente en los contenidos de la currícula de Arquitectura de Computadoras. Para lograr este objetivo, se plantean los siguientes objetivos específicos:

1. Estudiar y evaluar diferentes herramientas actuales de simulación destinadas a dar apoyo a la enseñanza de la arquitectura x86.
2. Construir una herramienta de apoyo para impartir los contenidos de la asignatura Arquitectura de Computadoras, para ello debe cumplir:
    - Una visión global de la estructura y funcionamiento de la computadora.
    - Generación y ejecución de programas en ensamblador.
    - Repertorio de instrucciones x86 reducido y habilitado progresivamente.
    - Simulación visual e interactiva de micropasos de instrucciones.
    - Gestión de interrupciones y comunicación con periféricos.
    - Medidas de rendimiento de ejecución de programas.

1.3 Metodología de desarrollo

Teniendo en cuenta los objetivos propuestos en la sección anterior, se pretende alcanzar los mismos a través de los pasos que se describen en esta sección.

1.3.1 Etapas de la investigación

a. Análisis bibliográfico. Se realizó una revisión continua de las publicaciones científicas y tecnológicas, libros e informes técnicos relacionados con el objeto de estudio.
b. Recopilación de simuladores. Se realizó un relevamiento del estado actual y las actualizaciones de los simuladores aplicados a la enseñanza de arquitectura de computadoras.
c. Estudio de los simuladores. En base a la documentación relevada de los simuladores se estudió en profundidad al menos 5 simuladores y se elaboró una comparativa de los simuladores seleccionados en cuanto a los contenidos que se imparten en la asignatura.
d. Construir el simulador. A través de métodos y técnicas de ingeniería de software se construyó un simulador de la arquitectura x86 donde abarque los aspectos más relevantes de la asignatura Arquitectura de Computadoras, permitiendo desarrollar los contenidos en una plataforma unificada, evitando así la pérdida de tiempo y dificultad que supone para el estudiante habituarse a diferentes entornos. Se utilizarán para ello técnicas formales de modelización y simulación, que facilitan un desarrollo modular y enfocan el esfuerzo en la definición del modelo de la arquitectura x86 más que en el protocolo de simulación, permitiendo además el escalado a entornos de ejecución paralelos o distribuidos sin necesidad de modificar el modelo de la arquitectura simulada.

1.4 Organización del documento

El resto de este documento se organiza de la siguiente manera: el capítulo 2 define formalmente las características y el set de instrucciones de la arquitectura. Luego, el capítulo 3 repasa y motiva el interesante rol que la simulación desde un punto de vista didáctico puede desempeñar para el dictado de la asignatura donde se abordan estos tópicos. El capítulo 4 comparativo de los simuladores estudiados según criterios preestablecidos. Finalmente, el capítulo 5 construcción de un simulador como soporte para el uso de la enseñanza y aprendizaje de arquitectura de computadora.

Capítulo 2

Estado del arte

En este capítulo se abordarán los aspectos fundamentales de la arquitectura de computadoras y su evolución, así como las tecnologías y filosofías que han dado forma al desarrollo de sistemas modernos. Se presentarán las arquitecturas más relevantes, el repertorio de instrucciones, y las filosofías de diseño, como CISC y RISC. Además, se discutirá la arquitectura x86, el lenguaje ensamblador y la simulación, con un enfoque especial en su aplicación educativa.

2.1 Arquitectura de computadoras

La arquitectura de computadoras abarca el diseño y la especificación de los componentes de un sistema informático que son visibles para el programador, como el conjunto de instrucciones, la organización de la memoria y los mecanismos de entrada/salida. Su principal objetivo es optimizar tanto el rendimiento como la eficiencia del sistema. Esta disciplina se extiende desde la interacción entre el hardware y el software hasta la integración de sistemas completos, siendo fundamental para el avance de tecnologías modernas y sostenibles [13], [14].

Comprender la arquitectura de una computadora implica tener un conocimiento profundo de atributos claves como el repertorio de instrucciones (por ejemplo, arquitecturas x86 o ARM), la capacidad de procesamiento (32 o 64 bits), los mecanismos de entrada/salida, las técnicas de direccionamiento de memoria (directo o segmentado) y la gestión de la jerarquía de memoria [22]. Además, aspectos como el paralelismo y la eficiencia energética son esenciales para mejorar el rendimiento de los sistemas, así como para optimizar el desarrollo de software y sistemas operativos modernos [13], [23].

Es esencial distinguir entre la **arquitectura de computadoras** y la **organización de computadoras**. Mientras que la arquitectura describe los componentes visibles

al programador y las abstracciones necesarias para desarrollar software eficiente, la organización se refiere a la implementación física de esa arquitectura. Esto incluye cómo están dispuestos y coordinados los elementos del hardware, como la memoria, las señales de control y las unidades de procesamiento, para cumplir con los objetivos de rendimiento y funcionalidad [24], [25].

El estudio de la arquitectura de computadoras no solo permite comprender el funcionamiento interno de los sistemas informáticos, sino que también facilita su adaptación y optimización para satisfacer las demandas de nuevas tendencias tecnológicas, como la computación en la nube, la inteligencia artificial y el Internet de las cosas. La arquitectura bien diseñada es el cimiento que permite a estas tecnologías alcanzar su máximo potencial, impulsando de manera significativa la innovación [13], [14].

Tener un conocimiento profundo de la arquitectura y la organización de los sistemas informáticos es vital para cualquier profesional en el campo de la informática. No solo habilita la creación de sistemas más eficientes y escalables, sino que también proporciona una base sólida para explorar áreas emergentes como la seguridad informática y los sistemas embebidos, sectores de creciente relevancia en la industria [23]. El dominio de la arquitectura de computadoras abre un amplio abanico de oportunidades profesionales, permitiendo a los expertos participar activamente en la evolución de la sociedad digital.

En resumen, el estudio de la arquitectura de computadoras es crucial no solo para entender cómo funcionan internamente los sistemas informáticos y resolver problemas de rendimiento, sino también para diseñar soluciones innovadoras en áreas clave como la inteligencia artificial, la ciberseguridad y los sistemas embebidos. Este conocimiento es esencial para abordar los retos tecnológicos del futuro y acceder a un mercado laboral que valora la capacidad de crear y optimizar sistemas cada vez más poderosos y eficientes [14].

2.2 Arquitecturas Von Neumann y Harvard

Para entender las arquitecturas modernas, es esencial conocer dos modelos históricos que han sentado las bases del diseño actual de sistemas informáticos: **Von Neumann** y **Harvard**. Estos paradigmas no solo forman el cimiento de muchas arquitecturas contemporáneas, sino que también ayudan a comprender sus diferencias, ventajas y limitaciones.

2.2.1 Arquitectura Von Neumann

La **arquitectura Von Neumann**, propuesta por John von Neumann en 1945, se caracteriza por utilizar un único espacio de memoria para almacenar tanto los datos

como las instrucciones, y un único bus para transferir ambos tipos de información. Esta simplicidad ha favorecido su adopción generalizada, pero también trae consigo una limitación importante: el **"cuello de botella de Von Neumann"**. Este fenómeno ocurre cuando el bus único se convierte en un factor restrictivo para la velocidad de procesamiento, especialmente en sistemas que requieren un acceso simultáneo a datos e instrucciones. Aunque esta arquitectura sigue siendo la base de muchas computadoras actuales, se han implementado optimizaciones y mejoras para mitigar sus limitaciones en los sistemas modernos [13], [14].

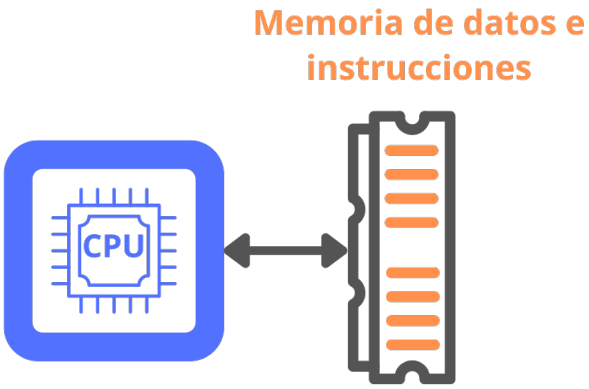

Figura 2.1: Arquitectura Von Neumann

2.2.2 Arquitectura Harvard

En contraste, la **arquitectura Harvard** separa físicamente las memorias de datos y de instrucciones, permitiendo que el procesador acceda a ambas de forma simultánea. Este diseño elimina el cuello de botella de Von Neumann, logrando un rendimiento más eficiente, lo que la convierte en una opción preferida para sistemas que requieren un alto desempeño, como los sistemas embebidos y los procesadores de señal digital (DSP). La arquitectura Harvard es especialmente adecuada para aplicaciones donde el acceso rápido y eficiente a los datos es crucial [24].

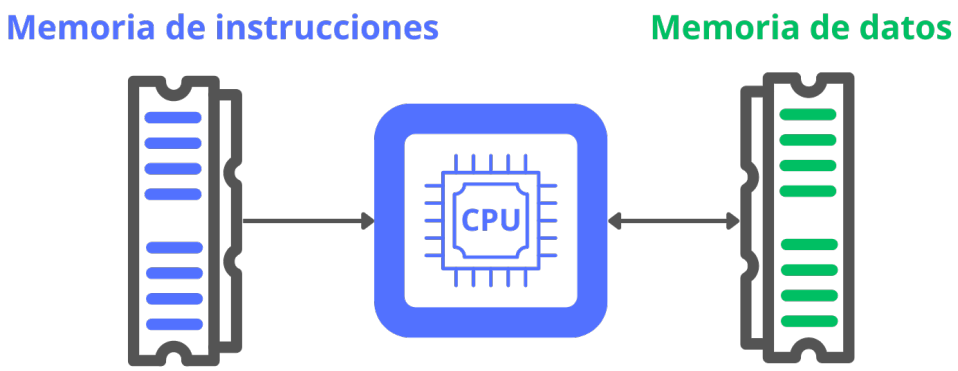

Figura 2.2: Arquitectura Harvard

2.2.3 Comparación y aplicaciones modernas

Si bien la simplicidad y flexibilidad de la arquitectura Von Neumann continúan siendo ventajas importantes para una amplia gama de sistemas, la arquitectura Harvard ha ganado popularidad en escenarios donde la eficiencia y la velocidad son prioritarias, como en dispositivos móviles y sistemas de tiempo real. Hoy en día, arquitecturas modernas como **x86**, **ARM** y **RISC-V** implementan una combinación de ambos modelos para maximizar tanto el rendimiento como la eficiencia energética [22].

Esta integración de los enfoques de Von Neumann y Harvard permite a los diseñadores aprovechar las mejores características de cada uno, adaptando los sistemas según las necesidades específicas de la aplicación. La dicotomía entre ambas arquitecturas sigue siendo clave en la evolución de los sistemas modernos, donde el rendimiento y la eficiencia son factores críticos para el desarrollo de nuevas tecnologías.

2.3 Tipos de arquitecturas

El análisis de diversas arquitecturas de computadoras es esencial para comprender sus respectivas ventajas y limitaciones en diferentes contextos de aplicación. Esta evaluación comparativa permite a los diseñadores y desarrolladores de sistemas seleccionar la arquitectura más adecuada para sus necesidades, considerando factores clave como la eficiencia energética, la complejidad del hardware y las aplicaciones específicas en las que se utilizará cada una.

2.3.1 Arquitectura x86

La arquitectura x86 ha dominado el mercado de las computadoras de escritorio y servidores durante décadas, gracias a su capacidad para ofrecer un alto rendimiento y su amplia compatibilidad con software existente. La complejidad de su conjunto de instrucciones (ISA) permite una mayor flexibilidad, aunque a costa de una mayor complejidad en el diseño del hardware. Su eficiencia energética es moderada, lo que la hace menos ideal para dispositivos móviles y más apropiada para entornos donde el rendimiento es la prioridad principal [13].

2.3.2 Arquitectura ARM

La arquitectura ARM es conocida por su alta eficiencia energética, característica que la ha hecho muy popular en dispositivos móviles, como smartphones y tablets, y en sistemas embebidos. ARM se basa en el paradigma de conjunto de instrucciones reducidas (RISC), lo que simplifica el diseño del hardware y reduce el consumo energético, como se explica en la sección 2.5.2. Aunque su rendimiento es moderado en comparación con x86, la arquitectura ARM es ideal para aplicaciones donde la eficiencia energética es fundamental [23].

2.3.3 Arquitectura MIPS

MIPS es otra arquitectura basada en RISC que ha sido ampliamente utilizada en sistemas embebidos y en la educación. Aunque ha perdido prominencia frente a otras arquitecturas como ARM, MIPS sigue siendo relevante en ciertos nichos debido a su simplicidad y efectividad. Su conjunto de instrucciones presenta una complejidad moderada, lo que permite un equilibrio entre rendimiento y consumo energético, aunque su adopción y soporte en la industria son más limitados [13].

2.3.4 Arquitectura RISC-V

RISC-V es una arquitectura emergente que ha atraído la atención global debido a su conjunto de instrucciones abierto y libre, lo que permite a desarrolladores y fabricantes adaptarla y personalizarla según sus necesidades. Al igual que ARM, RISC-V se basa en el paradigma RISC, lo que le otorga una alta eficiencia energética y una baja complejidad de hardware. Su flexibilidad la convierte en una opción atractiva para la investigación, la educación y aplicaciones embebidas. Además, RISC-V ofrece una compatibilidad creciente gracias a su adopción por parte de la comunidad global de desarrolladores [26].

2.3.5 Compararativa de arquitecturas

Cada una de estas arquitecturas ofrece un conjunto de características distintivas que las hace más o menos adecuadas para diferentes aplicaciones. La elección de la arquitectura correcta puede tener un impacto significativo en el éxito de un proyecto, ya sea en términos de rendimiento, eficiencia energética o compatibilidad [23].

Cuadro 2.1: Comparación de Arquitecturas

Característica	x86	ARM	MIPS	RISC.V
Eficiencia Energética	Moderada	Alta	Moderada	Alta
Complejidad ISA	Alta	Baja	Moderada	Baja
Rendimiento	Alto	Moderado	Moderado	Variable
Compatibilidad	Alta (hacia atrás)	Moderada	Moderada	Alta
Áreas de Aplicación	Escritorio, servidores	Dispositivos móviles	Sistemas embebidos	Investigación, embebidos

2.4 Repertorio de instrucciones

El repertorio de instrucciones, conocido como **ISA (Instruction Set Architecture)**, define el conjunto de operaciones que un procesador puede ejecutar y cómo se codifican dichas operaciones. Este conjunto incluye instrucciones aritméticas, lógicas, de control y de manipulación de datos, así como los modos de direccionamiento y formatos de las instrucciones. El diseño de un ISA tiene un impacto considerable en el rendimiento, la eficiencia energética y la flexibilidad de la arquitectura de un procesador [13], [14], [22].

Entre las características clave que deben considerarse en el diseño de un repertorio de instrucciones están las siguientes [13]:

- **Tipos de operandos**: Se refiere a los diversos tipos de datos que las instrucciones pueden manipular, como enteros, números en punto flotante, caracteres o direcciones de memoria. Es crucial diseñar un repertorio de instrucciones que soporte eficientemente diferentes tipos de operando.
- **Tipos de operaciones**: Incluye las operaciones que el procesador puede realizar, como aritmética (suma, resta), lógica (AND, OR), control (saltos, llamadas a subrutinas) y operaciones de manipulación de datos (movimiento de datos, almacenamiento, carga).

- **Formato de las instrucciones**: La estructura de una instrucción determina la longitud, el número de operandos y los modos de direccionamiento, lo que impacta directamente en la complejidad y eficiencia del procesador.
- **Modos de direccionamiento**: Definen cómo se especifican los operandos en las instrucciones. Modos comunes incluyen inmediato, directo, indirecto y relativo, cada uno ofreciendo diferentes ventajas en términos de flexibilidad y eficiencia.

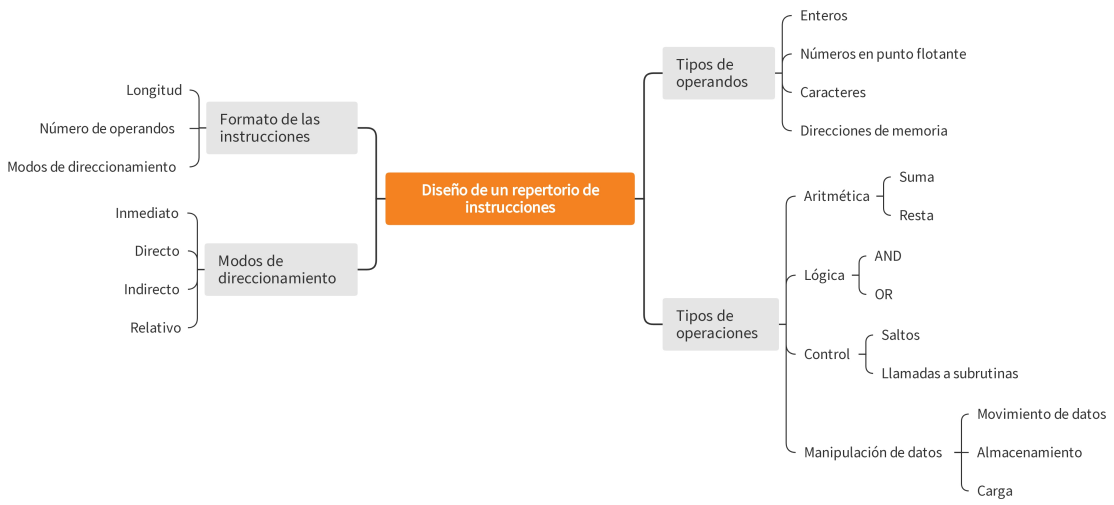

Figura 2.3: Características repertorio de instrucciones

2.4.1 Modos de direccionamiento

Los modos de direccionamiento son esenciales para especificar cómo la CPU accede a los datos necesarios para ejecutar una instrucción [13], [14]. Los modos más comunes incluyen:

a) **Inmediato**: El operando está directamente incluido en la instrucción, lo que permite un acceso rápido a valores constantes, generalmente pequeños. Este modo es eficiente para operaciones simples, pero está limitado a operandos de tamaño reducido.
b) **Directo**: La instrucción contiene la dirección de memoria del operando. Este modo es fácil de usar y entender, pero tiene la desventaja de que el rango de direcciones accesibles está limitado por el tamaño de la instrucción.
c) **Indirecto**: La instrucción apunta a una dirección que, a su vez, contiene la dirección real del operando. Este modo ofrece una mayor flexibilidad al ampliar el rango de direcciones, aunque introduce una penalización en tiempo debido al acceso adicional a la memoria.

d) **Registro**: El operando está ubicado en uno de los registros del procesador, lo que permite un acceso extremadamente rápido. Este modo es muy eficiente, ya que evita los accesos a la memoria, pero está limitado por el número de registros disponibles.
e) **Registro Indirecto**: Similar al modo indirecto, pero en este caso la dirección del operando está almacenada en un registro. Esto combina la rapidez del acceso a registros con la flexibilidad del direccionamiento indirecto.
f) **Con Desplazamiento**: Este modo combina una dirección base con un valor de desplazamiento, lo que resulta muy útil para trabajar con estructuras de datos como arrays. Permite un acceso eficiente a elementos contiguos en memoria.
g) **Pila**: El operando se encuentra en la parte superior de la pila, y su dirección es calculada en base al puntero de la pila. Este modo es fundamental para la gestión de llamadas a subrutinas y el paso de parámetros en muchos lenguajes de programación.

Estos modos se ilustran en la figura 2.4 según [14]:

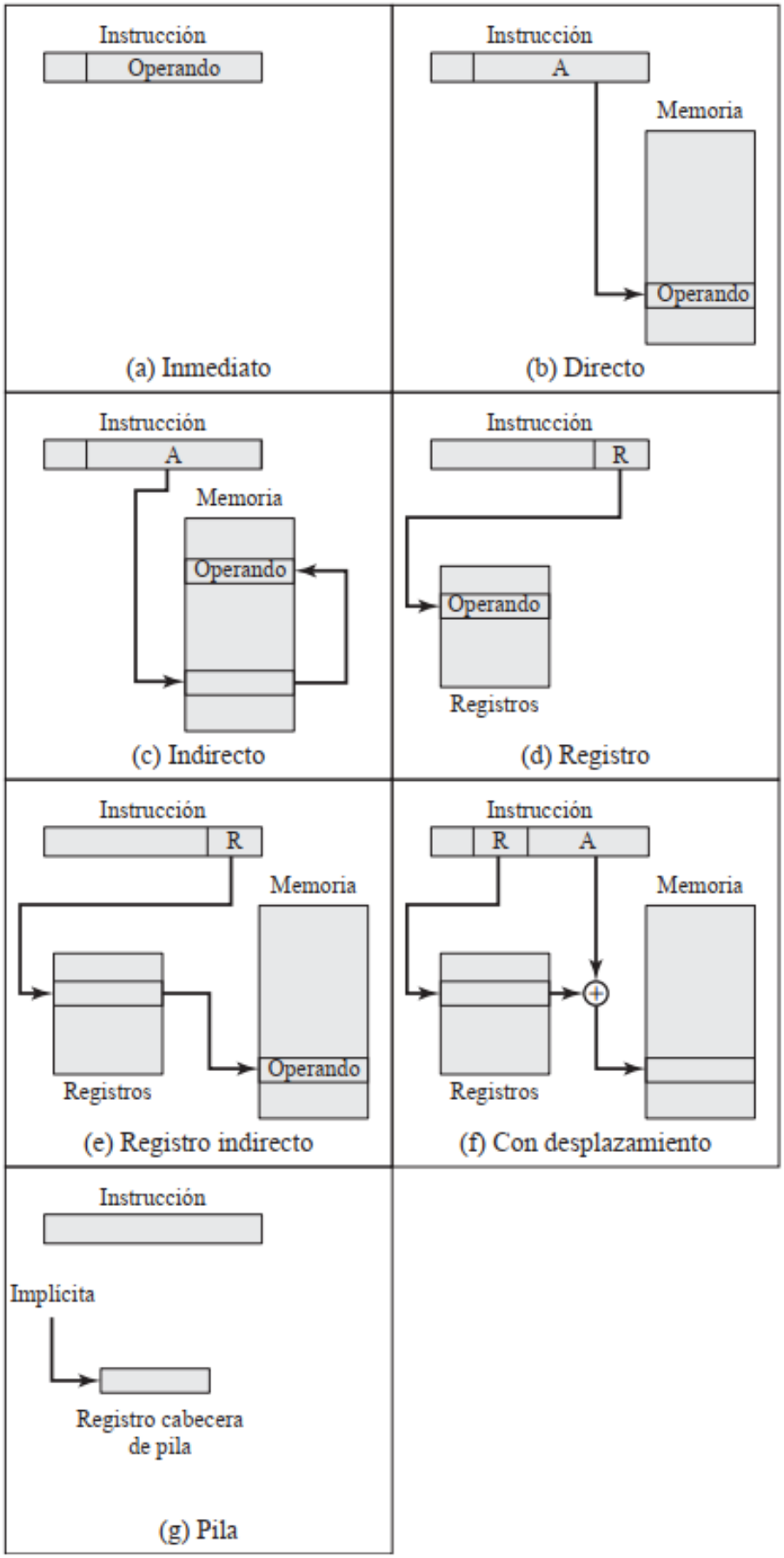

Figura 2.4: Modos de direccionamiento

- A = contenido de un campo de dirección en la instrucción
- R = contenido de un campo de dirección en la instrucción que referencia a un registro
- EA = dirección real (efectiva) de la posición que contiene el operando que se referencia

La tabla 2.2 indica el cálculo de la dirección realizado para cada modo de direccionamiento.

Cuadro 2.2: Modos de direccionamiento básicos

Modo	Algoritmo	Principal.ventaja	Principal.desventaja
Inmediato	Operando ← A	No referencia a memoria	Operando de magnitud limitada
Directo	EA ← A	Es sencillo	Espacio de direcciones limitado
Indirecto	EA ← (A)	Espacio de direcciones grande	Referencias múltiples a memoria
Registro	EA ← R	No referencia a memoria	Número limitado de registros
Indirecto con registro	EA ← (R)	Espacio de direcciones grande	Referencia extra a memoria
Con desplazamiento	EA ← A + (R)	Flexibilidad	Complejidad
Pila	EA ← cabecera de la pila	No referencia a memoria	Aplicabilidad limitada

2.4.2 Formato de las instrucciones

El **formato de las instrucciones** establece cómo se estructuran las instrucciones que un procesador debe ejecutar, lo que incluye su longitud, la cantidad de operandos y los campos adicionales como el código de operación (**opcode**) [13], [24]. Este formato no solo afecta la rapidez con la que las instrucciones pueden ser decodificadas, sino también el grado de flexibilidad del conjunto de instrucciones y el aprovechamiento de los recursos del procesador.

- **Longitud de la instrucción**: Las instrucciones pueden tener una longitud fija o variable, y esta decisión afecta tanto al diseño como al rendimiento del procesador. Las instrucciones de longitud fija simplifican el proceso de decodificación, ya que todas tienen el mismo tamaño, lo que facilita el diseño del hardware. Sin embargo, esto puede resultar en un uso ineficiente de la memoria cuando

se utilizan instrucciones más simples que no requieren tanto espacio. En contraste, las instrucciones de longitud variable permiten un uso más eficiente de la memoria, ya que el tamaño de cada instrucción puede ajustarse a las necesidades de la operación. No obstante, esta flexibilidad aumenta la complejidad en la decodificación, ya que el procesador debe determinar primero la longitud de la instrucción antes de ejecutarla.

- **Cantidad de operandos**: Las instrucciones pueden trabajar con diferentes cantidades de operandos según la arquitectura del procesador, variando desde cero hasta tres o más. A mayor número de operandos, las instrucciones son más versátiles, permitiendo realizar operaciones más complejas en una única instrucción, lo que reduce el número total de instrucciones necesarias. Sin embargo, un mayor número de operandos también implica un aumento en la complejidad del procesador, ya que se necesitan más campos en la instrucción para especificar los operandos y más registros o direcciones de memoria para manejar los datos, lo cual puede incrementar la longitud de las instrucciones y el esfuerzo de decodificación.
- **Campos de instrucción**: Entre los campos clave que componen una instrucción, el **opcode** es el más relevante, ya que especifica la operación que el procesador debe ejecutar, como suma, resta o saltos condicionales. Además, las instrucciones pueden incluir otros campos para indicar los operandos, los modos de direccionamiento, y en arquitecturas más avanzadas, flags de condición o características de optimización, como la predicción de saltos o la ejecución fuera de orden. La disposición y el tamaño de estos campos determinan cuántas y qué tipo de operaciones puede realizar el procesador en un ciclo de reloj, afectando directamente su rendimiento global. En arquitecturas más complejas, los campos adicionales permiten un mayor grado de optimización, mejorando el rendimiento en aplicaciones específicas, aunque a costa de un mayor esfuerzo en el diseño y la implementación del hardware.

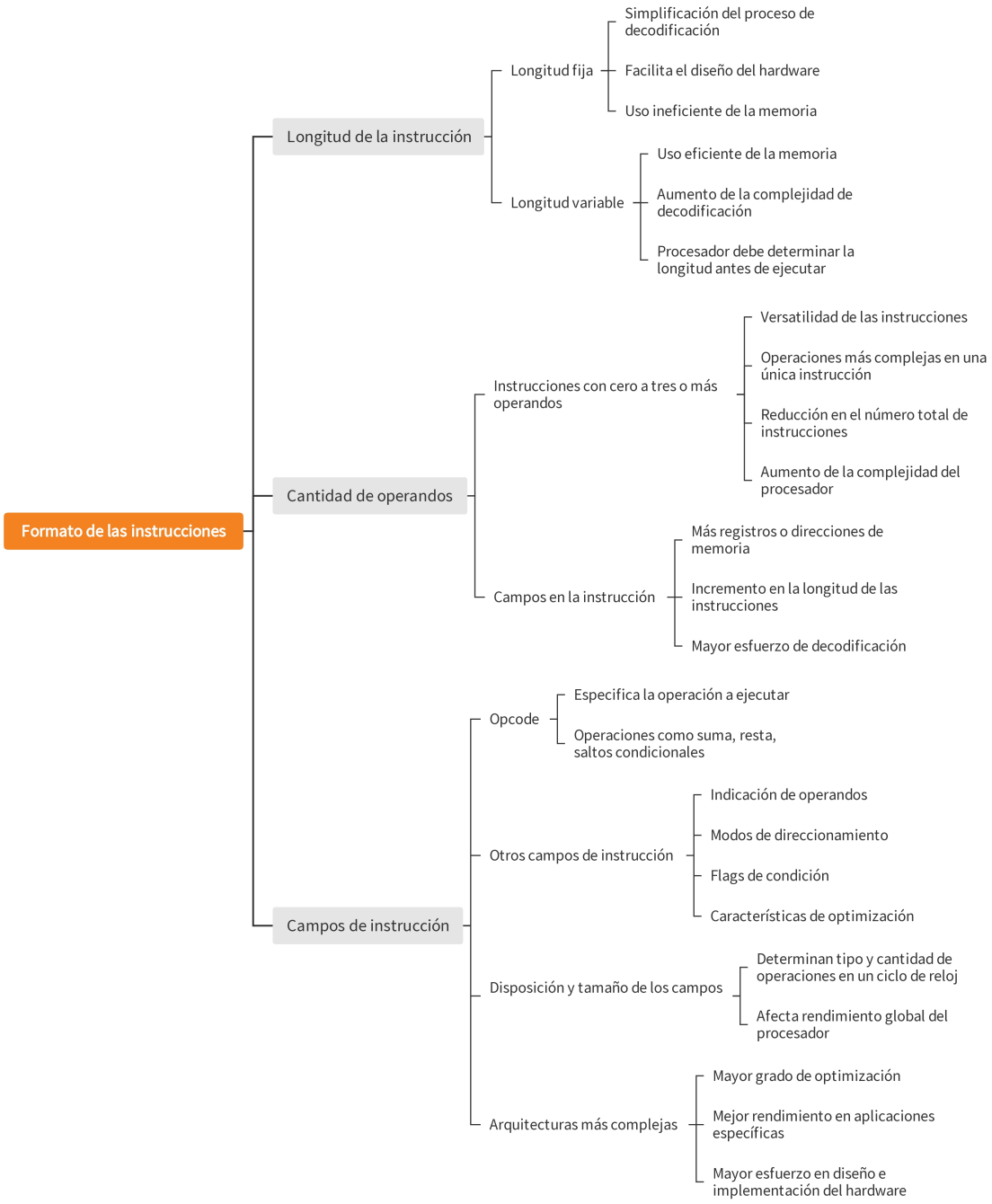

Figura 2.5: Formato de instrucciones

2.5 Filosofías CISC y RISC

Las arquitecturas de repertorio de instrucciones son componentes cruciales en el diseño de procesadores. Dos de las filosofías de diseño más influyentes en este ámbito son

CISC (Complex Instruction Set Computing) y **RISC (Reduced Instruction Set Computing)**. Mientras que CISC busca reducir la cantidad de instrucciones necesarias para realizar una tarea compleja, RISC simplifica el conjunto de instrucciones para optimizar la velocidad y la eficiencia energética. En esta sección se exploran estos dos enfoques y sus implicaciones en el diseño de procesadores [13], [23].

2.5.1 CISC

Las arquitecturas **CISC**, como la **x86**, tienen como objetivo minimizar el número de instrucciones necesarias para llevar a cabo operaciones complejas. Esto se logra mediante la implementación de instrucciones que pueden realizar varias operaciones en un solo ciclo de instrucción, reduciendo así la cantidad de código que debe escribir un programador. Sin embargo, este enfoque conlleva un costo: la **decodificación** y **ejecución** de instrucciones CISC requieren hardware más complejo. Además, las instrucciones de longitud variable, comunes en CISC, pueden aumentar el tiempo de decodificación, generando cuellos de botella en el pipeline.

Por ejemplo, los procesadores **x86** han evolucionado hacia una arquitectura híbrida, utilizando microcódigo para descomponer las instrucciones CISC en operaciones más simples, similares a las de una arquitectura RISC. Esto mejora la eficiencia de ejecución en ciertos contextos, aunque sigue siendo una arquitectura más costosa en términos de diseño y consumo energético [23].

2.5.2 RISC

Por otro lado, las arquitecturas **RISC**, como **ARM** y **MIPS**, se centran en un conjunto de instrucciones más reducido y de longitud fija, lo que simplifica la decodificación y permite la ejecución de instrucciones en un solo ciclo de reloj. Esta simplicidad también facilita la implementación de técnicas avanzadas de **pipelining** y **predicción de ramas**, mejorando el rendimiento en operaciones básicas. A nivel de hardware, RISC favorece la **optimización** y el uso eficiente de la energía, lo que resulta crucial en dispositivos móviles y embebidos [13].

La filosofía de diseño RISC ha permitido que procesadores como **ARM** dominen el mercado de dispositivos móviles, donde la eficiencia energética es primordial. El enfoque de RISC en operaciones simples y repetitivas, con un bajo CPI (ciclos por instrucción), ha sido un factor clave en esta adopción [13].

2.5.3 Impacto del formato de instrucciones entre CISC y RISC

En las arquitecturas **RISC**, todas las instrucciones tienen la misma longitud, lo que simplifica la decodificación y permite al procesador mantener un flujo continuo de instrucciones a través del pipeline. Este enfoque no solo reduce la latencia, sino que también mejora la predictibilidad del rendimiento, lo cual es esencial para arquitecturas altamente escalables y eficientes. Además, este formato simplificado permite una mejor **utilización de la memoria caché**, ya que las instrucciones ocupan menos espacio en la memoria y permiten un acceso más rápido.

Por el contrario, las arquitecturas **CISC**, como **x86**, utilizan un **formato de longitud variable** para proporcionar flexibilidad en las instrucciones. Esto permite que las arquitecturas CISC ofrezcan una gama más amplia de operaciones con menos líneas de código. Sin embargo, esta flexibilidad tiene un costo: las instrucciones de longitud variable requieren más tiempo de decodificación y complican la implementación del pipeline, lo que puede afectar negativamente al rendimiento [24].

Por ejemplo, en un procesador x86, la decodificación de instrucciones de longitud variable puede ser un cuello de botella, especialmente cuando el pipeline se interrumpe debido a errores de predicción de ramas. Aunque se pueden mitigar estos problemas mediante técnicas avanzadas como la predicción dinámica de saltos y el prefetching, el impacto en el rendimiento sigue siendo significativo.

2.5.4 Ejemplos prácticos de modos de direccionamiento entre CISC y RISC

Los **modos de direccionamiento** permiten a los procesadores acceder a datos de manera eficiente y flexible [14]. A continuación se incluyen ejemplos prácticos de cómo estos modos se utilizan en las arquitecturas **x86**, **ARM** y **MIPS**, y cómo afectan el rendimiento.

Modo inmediato vs. indirecto

En el modo de **direccionamiento inmediato**, el valor del operando está incrustado directamente en la instrucción, lo que permite un acceso rápido y eficiente. Por ejemplo, en ARM, este modo es común para operaciones aritméticas simples, donde se requiere velocidad en lugar de flexibilidad.

```
MOV R0, #10  ; Carga el valor inmediato 10 en el registro R0
```

En contraste, el modo de direccionamiento indirecto utiliza una dirección almacenada en un registro para acceder a la memoria, lo que añade flexibilidad a costa de un mayor tiempo de acceso. En la arquitectura x86, este modo es frecuente en operaciones complejas de acceso a memoria.

```
MOV EAX, [EBX]  ; Carga en EAX el valor almacenado en la dirección apuntada por EBX
```

Impacto en el rendimiento

El uso de modos de direccionamiento como el indirecto introduce una mayor latencia debido al acceso adicional a la memoria. En arquitecturas como x86, este impacto puede mitigarse parcialmente mediante el uso de cachés y técnicas de prefetching, pero sigue siendo un factor clave en el rendimiento general del sistema. En arquitecturas RISC como ARM o MIPS, se priorizan los modos de direccionamiento simples, lo que permite un acceso más rápido a los operandos y reduce la latencia del pipeline [14].

La elección entre CISC y RISC, así como el diseño del formato de instrucciones y los modos de direccionamiento, afecta profundamente el rendimiento y la eficiencia de un procesador. Mientras que CISC ofrece flexibilidad y un repertorio de instrucciones amplio a costa de una mayor complejidad, RISC prioriza la simplicidad y la velocidad. A medida que las arquitecturas de procesadores evolucionan, se observa una convergencia de ambas filosofías, con arquitecturas modernas que integran características de ambos enfoques para maximizar el rendimiento y la eficiencia energética. Estos conceptos son fundamentales para entender el estado actual del diseño de procesadores y su evolución futura.

Cuadro 2.3: Comparativa entre CISC y RISC

Aspecto	CISC	RISC
Objetivo principal	Minimizar el número de instrucciones para operaciones complejas	Simplificar el conjunto de instrucciones para optimizar velocidad y eficiencia energética
Tipo de instrucciones	Instrucciones complejas, longitud variable	Instrucciones simples, longitud fija
Decodificación y ejecución	Requiere hardware más complejo, posibles cuellos de botella en el pipeline	Decodificación más sencilla, facilita el uso de técnicas avanzadas como pipelining
Longitud de instrucciones	Longitud variable, puede aumentar el tiempo de decodificación	Longitud fija, simplifica la decodificación y mejora la predictibilidad del rendimiento
Eficiencia energética	Menor eficiencia energética en comparación con RISC	Mayor eficiencia energética, especialmente en dispositivos móviles

Aspecto	CISC	RISC
Modos de direccionamiento	Flexibilidad a costa de mayor latencia	Acceso más rápido a los operandos, menor latencia

2.6 Arquitectura x86

La arquitectura x86, una de las más influyentes y ampliamente utilizadas en el ámbito de las computadoras de escritorio y servidores, comenzó su desarrollo en 1978 con el lanzamiento del procesador Intel 8086, que introdujo una arquitectura de 16 bits. La arquitectura x86 evolucionó significativamente con el Intel 80386 en 1985, marcando el inicio de la era de 32 bits. En 2003, AMD lanzó la arquitectura AMD64, extendiendo x86 a 64 bits, lo que permitió un mayor acceso a la memoria y un mejor rendimiento en aplicaciones intensivas. Intel adoptó estas innovaciones, consolidando la arquitectura x86 como una de las más versátiles y potentes del mercado [14, p. @intel_64_2016], [16], [17].

2.6.1 Evolución de la arquitectura x86

La retrocompatibilidad de la arquitectura x86 ha sido un factor determinante en su éxito, permitiendo que aplicaciones de 16, 32 y 64 bits se ejecuten en el mismo sistema. Esta característica ha asegurado la continuidad y protección de las inversiones en software y sistemas operativos desarrollados para x86.

La retrocompatibilidad de la arquitectura x86 ha sido un factor determinante en su éxito, permitiendo que aplicaciones de 16, 32 y 64 bits se ejecuten en el mismo sistema. Esta característica ha asegurado la continuidad y protección de las inversiones en software y sistemas operativos desarrollados para x86.

Cuadro 2.4: Hitos procesadores x86

Procesador	Año.de.Lanzamiento	Número.de.Bits	Nuevas.Características
Intel 8086	1978	16	Arquitectura inicial
Intel 80386	1985	32	Memoria virtual
AMD64	2003	64	Extensiones de 64 bits

La evolución de la arquitectura x86 ha estado marcada por hitos importantes que han impulsado la informática hacia nuevas alturas. Tras el Intel 8086, el lanzamiento del Intel 80286 en 1982 introdujo modos de operación adicionales que mejoraron la eficiencia y el manejo de memoria. En 1989, el Intel 80486 incorporó una unidad

de punto flotante integrada y una mejor caché, aumentando significativamente el rendimiento.

La serie Pentium, iniciada en 1993, llevó la arquitectura x86 a nuevos niveles de rendimiento y eficiencia, con características avanzadas como la ejecución superescalar y la predicción de saltos. El Pentium Pro en 1995 mejoró la arquitectura con ejecución fuera de orden y una caché L2 integrada.

En la década de 2000, la arquitectura x86 se adaptó a las demandas de la computación moderna con la introducción del Intel Core, optimizando el rendimiento y la eficiencia energética. AMD también fue crucial con su serie Athlon y la introducción de AMD64, llevando la arquitectura x86 a 64 bits, permitiendo un mayor acceso a la memoria y mejorando el rendimiento en aplicaciones intensivas.

La arquitectura x86 ha tenido un impacto profundo en el desarrollo de software. Los sistemas operativos populares como Windows y Linux están optimizados para x86, lo que ha influido en el desarrollo y la optimización de aplicaciones para esta arquitectura.

- **Influencia en el desarrollo de software**: Los desarrolladores de software han trabajado estrechamente con las características de la arquitectura x86 para optimizar el rendimiento de sus aplicaciones. Esto incluye el uso de instrucciones específicas de x86 y la optimización para cachés y pipelines.
- **Compatibilidad y soporte**: La compatibilidad hacia atrás de x86 ha permitido la continuidad de aplicaciones y sistemas operativos, protegiendo las inversiones en software y facilitando las actualizaciones.
- **Ecosistema de desarrollo**: Un amplio ecosistema de herramientas de desarrollo, bibliotecas y frameworks ha sido construido alrededor de la arquitectura x86, facilitando el desarrollo de aplicaciones de alto rendimiento y su depuración.

Cuadro 2.5: Línea de Tiempo de la Evolución de la Arquitectura x86

Año	Procesador	Innovación_Principal
1978	Intel 8086	Introducción de la arquitectura x86, 16 bits
1982	Intel 80286	Modos de operación adicionales
1985	Intel 80386	Arquitectura de 32 bits, memoria virtual
1989	Intel 80486	Unidad de punto flotante integrada, mejor caché
1993	Intel Pentium	Ejecución superescalar, predicción de saltos
1995	Intel Pentium Pro	Ejecución fuera de orden, caché L2 integrada
2003	AMD64	Extensiones a 64 bits, mayor acceso a memoria
2006	Intel Core	Optimización de rendimiento y eficiencia energética

2.6.2 Repertorio de instrucciones x86

La arquitectura x86 se caracteriza por su notable **complejidad y flexibilidad**, reflejada en su extenso y variado repertorio de instrucciones. A diferencia de las arquitecturas RISC, que se distinguen por instrucciones de **longitud fija** y **decodificación sencilla**, el conjunto de instrucciones x86 es de **longitud variable**, lo que introduce una carga significativa en el proceso de decodificación [13]. Esta flexibilidad ha permitido que x86 evolucione y se adapte a una amplia gama de aplicaciones, desde dispositivos de bajo consumo hasta servidores de alto rendimiento. Sin embargo, también ha incrementado su **complejidad** tanto en términos de implementación como de eficiencia operativa.

Estructura de una Instrucción x86

Las instrucciones en la arquitectura x86 son más complejas debido a la variedad de componentes opcionales que pueden formar parte de ellas. Una instrucción típica puede estar compuesta por los siguientes elementos [14]:

- **Prefijos**: Algunas instrucciones pueden incluir uno o más bytes de prefijo, que alteran la operación de la instrucción principal. Por ejemplo, el prefijo `0x66` se utiliza para cambiar el tamaño del operando de 32 bits a 16 bits. Aunque los prefijos proporcionan una gran flexibilidad, también añaden complejidad a la decodificación, ya que el procesador debe interpretarlos antes de ejecutar la instrucción [23]. Esto contrasta con arquitecturas como MIPS o ARM, donde la longitud fija de las instrucciones simplifica la decodificación.
- **Código de Operación (Opcode)**: El **opcode** indica la operación que se va a realizar. En x86, el opcode `0x89` corresponde a la instrucción `MOV`, que mueve datos entre registros o entre un registro y la memoria. Debido a la vasta cantidad de instrucciones que soporta x86, el conjunto de opcodes es mucho más amplio en comparación con las arquitecturas RISC, que se enfocan en un conjunto más reducido y optimizado [22].
- **Modificadores de Dirección (ModR/M y SIB)**: Estos campos especifican los registros o las direcciones de memoria involucrados en la operación. El byte **ModR/M** es clave para definir qué registros participan en la instrucción, como `0xC1`, que indica el uso del registro `ECX`. El byte **SIB** (Scale, Index, Base) añade flexibilidad en el direccionamiento, permitiendo combinaciones complejas de registros y escalas de índices para acceso eficiente a memoria, algo particularmente útil en operaciones de matrices y estructuras de datos más complejas [14].
- **Desplazamiento y/o Inmediato**: Dependiendo del tipo de instrucción, esta puede incluir un valor de desplazamiento para direccionamiento indirecto o un valor inmediato como operando. Un desplazamiento de `0x10`, por ejemplo, representa un offset de 16 bytes desde la dirección base. La posibilidad de usar

direccionamiento indirecto e inmediato incrementa la flexibilidad en el manejo de datos, aunque también añade complejidad a la ejecución [24].

Una característica distintiva de x86 es la **variabilidad en la longitud de sus instrucciones**, que puede oscilar entre **1 y 15 bytes**. Esta variabilidad hace que la decodificación sea mucho más complicada que en arquitecturas como ARM o MIPS, donde las instrucciones tienen una longitud constante [22]. En x86, el procesador debe identificar y decodificar los múltiples componentes de la instrucción, lo cual requiere más recursos y etapas en el pipeline de procesamiento.

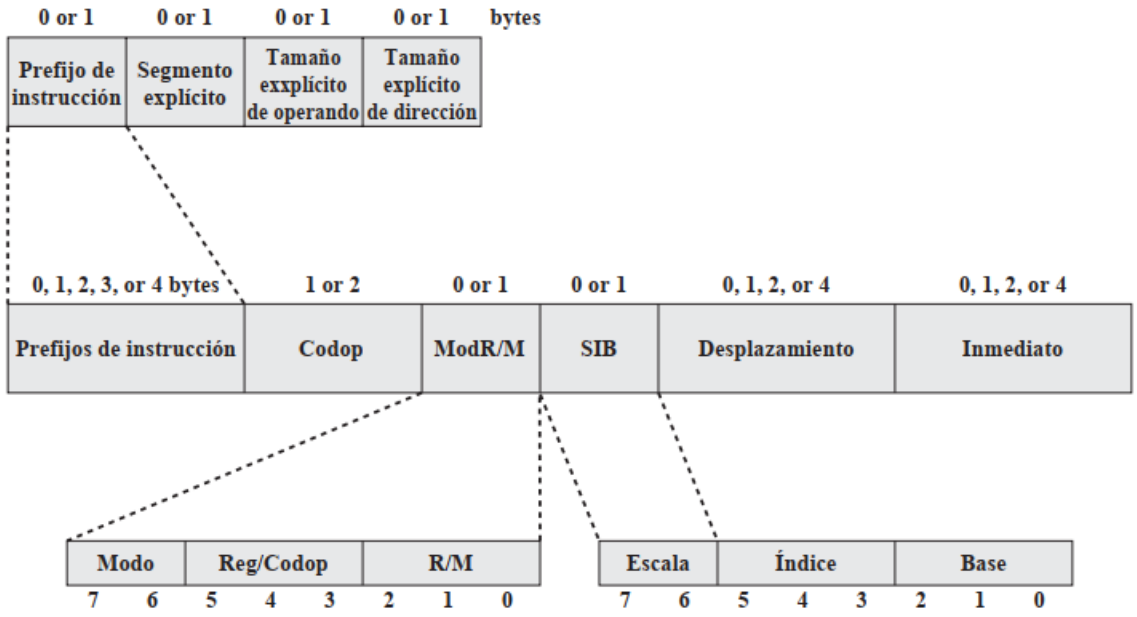

Figura 2.6: Formato de instrucciones del Pentium x86

Ejemplo de Instrucción x86:

```
MOV AX, [BX+SI+16]
```

En este caso, el opcode `MOV` se traduce a un byte específico, mientras que el uso de registros y desplazamientos se codifica mediante los bytes **ModR/M** y posiblemente **SIB**. Este ejemplo ilustra cómo una instrucción aparentemente compacta puede expandirse a múltiples bytes en memoria, y cómo el procesador debe descomponer estos bytes para ejecutar la operación correctamente. Este tipo de instrucciones, aunque versátiles, requieren de un pipeline avanzado y técnicas de optimización, como la paralelización y la predicción de saltos, para ser ejecutadas de manera eficiente en procesadores modernos [23].

2.7 Lenguaje ensamblador

Un procesador solo puede interpretar y ejecutar instrucciones en **lenguaje máquina**, que son secuencias de números binarios almacenados en la memoria. Estas instrucciones son leídas y ejecutadas directamente por el procesador. Si un programador quisiera escribir código en lenguaje máquina, tendría que especificar manualmente las secuencias de ceros y unos para cada operación, respetando las estructuras de memoria y los modos de direccionamiento del procesador. Este proceso es extremadamente tedioso y altamente propenso a errores, especialmente al momento de modificar el código, ya que requeriría traducir nuevamente las secuencias binarias a instrucciones comprensibles [27].

Para hacer este proceso más accesible, se desarrolló el **lenguaje ensamblador**, un lenguaje de programación de bajo nivel que permite a los programadores escribir instrucciones más legibles mediante el uso de mnemónicos simbólicos. A diferencia del lenguaje máquina, que se basa en secuencias binarias, el ensamblador emplea símbolos que representan directamente las instrucciones que el procesador debe ejecutar. Cada arquitectura de procesador cuenta con su propio lenguaje ensamblador, generalmente definido por el fabricante del hardware y adaptado a la arquitectura específica del procesador [14].

2.7.1 Ensamblador

El ensamblador, como programa, es el encargado de traducir las instrucciones simbólicas (mnemónicos) escritas por los programadores a **lenguaje máquina**, es decir, a las secuencias de números binarios que el procesador puede interpretar y ejecutar. Esta traducción se realiza de manera casi directa, manteniendo una correspondencia uno a uno entre las instrucciones en ensamblador y las instrucciones en lenguaje máquina [14]. A diferencia de los lenguajes de programación de alto nivel, como C o Python, donde cada línea de código generalmente se traduce en múltiples instrucciones máquina, el ensamblador ofrece una relación más cercana entre el código fuente y las instrucciones ejecutadas.

Ensambladores x86

El código fuente escrito en ensamblador debe ser procesado por un **ensamblador**, un programa que convierte el código simbólico en lenguaje máquina, permitiendo su ejecución por la computadora. En el contexto de la arquitectura **x86**, existen diversos ensambladores, como **TASM (Turbo Assembler)** [28], **MASM (Microsoft Macro Assembler)** [29] y **NASM (Netwide Assembler)** [30]. Cada uno de estos ensambladores tiene características y sintaxis particulares, pero todos comparten el

Cuadro 2.6: Comparativa de Ensambladores x86

Característica	TASM	MASM
Desarrollador	Borland	Microsoft
Año de lanzamiento	1985	1981
Sistema operativo	MS-DOS, Windows	MS-DOS,
Sintaxis	Sintaxis similar a la de Intel, con extensiones propias	Sintaxis de
Soporte de macros	Extensas capacidades de macros y directivas	Extensas c
Compatibilidad	Compatible con versiones antiguas de x86	Compatibl
Capacidades adicionales	Integración con herramientas Borland	Integración
Licencia	Comercial	Comercial
Uso actual	Menos común, pero todavía en uso en entornos heredados	Ampliamen

objetivo de traducir el código ensamblador en secuencias binarias ejecutables por el procesador x86 [31].

Ensambladores x86

2.8 Simulación

La simulación es una herramienta fundamental en diversos campos como la medicina, el ámbito militar, el entretenimiento y la educación. Su utilidad radica en su capacidad para facilitar la comprensión de sistemas complejos, permitir la generación de hipótesis, realizar análisis predictivos y explorar escenarios hipotéticos.

Según Banks [2], la simulación se define como el proceso de replicar el comportamiento de un sistema a lo largo del tiempo mediante la creación de un modelo conceptual que refleje sus características y dinámicas. Este modelo simulado evoluciona con el tiempo, proporcionando una representación detallada de cómo se desarrolla el sistema real [2][4].

La capacidad de replicar y analizar sistemas complejos sin necesidad de intervención directa convierte a la simulación en una metodología indispensable para ingenieros, diseñadores y gerentes en el entorno digital actual. Permite evaluar el rendimiento de sistemas, prever su comportamiento en diferentes escenarios y optimizar su diseño antes de su implementación. Con los avances tecnológicos, la simulación se ha consolidado como una herramienta crucial para ingenieros, docentes, diseñadores y gerentes, debido a que la complejidad intrínseca de los sistemas informáticos dificulta su comprensión y encarece su desarrollo sin técnicas de simulación [3][20].

2.8.1 Aplicaciones de la Simulación en la Industria

En la industria automotriz, la simulación es fundamental para el diseño y prueba de sistemas de seguridad, como airbags y frenos. Utilizando un modelo virtual del automóvil y sus componentes, es posible realizar pruebas de colisión y evaluar el rendimiento de los sistemas de seguridad sin recurrir a costosas pruebas físicas. Además, la simulación permite optimizar el diseño de motores, analizar el flujo aerodinámico y prever el comportamiento de los materiales en condiciones extremas [32].

En el campo de la aviación, la simulación se emplea para entrenar pilotos en simuladores de vuelo que replican condiciones reales sin riesgos. También se utiliza en el diseño de aeronaves para evaluar la aerodinámica y el rendimiento de nuevos diseños bajo diversas condiciones de vuelo. Estos ejemplos demuestran cómo la simulación puede reducir costos, mejorar la seguridad y acelerar el desarrollo de productos complejos [33].

2.9 Simulación en la educación

En el ámbito educativo, la simulación se destaca como una herramienta poderosa para enseñar conceptos complejos y fomentar el aprendizaje activo. Los simuladores permiten a los estudiantes interactuar con sistemas virtuales y experimentar con escenarios realistas, lo que facilita la comprensión de conceptos abstractos y la aplicación de conocimientos teóricos en situaciones prácticas [5].

Para superar las limitaciones de los métodos tradicionales de enseñanza, como el uso de pizarras, libros de texto y diapositivas, las herramientas de simulación han demostrado ser fundamentales. La integración de nuevas tecnologías como recurso didáctico facilita el aprendizaje, ya que permiten a los alumnos relacionar conceptos abstractos con situaciones reales en contextos que imitan aspectos de la realidad. Este enfoque pedagógico promueve la identificación de problemas y el desarrollo de habilidades a través del trabajo exploratorio, la inferencia y el aprendizaje por descubrimiento [6].

En resumen, la simulación no solo mejora la comprensión de los estudiantes, sino que también fomenta la experimentación y promueve un aprendizaje más activo y participativo.[5].

2.9.1 El rol de la simulación en la enseñanza de Arquitectura de Computadoras

La asignatura **Arquitectura de Computadoras** de la carrera Licenciatura en Sistemas tiene por objetivos: - Comprender la estructura y funcionamiento de las com-

putadoras.
- Conocer las diferentes arquitecturas de sistemas microprocesadores.
- Evaluar medidas de rendimiento y comparar arquitecturas.
- Analizar el impacto de la tecnología de las computadoras en contextos sociales y económicos.

Transmitir los fundamentos teóricos de la organización y arquitectura interna de las computadoras puede ser un desafío debido a la complejidad de los procesos involucrados. Esto requiere que los alumno realicen un alto nivel de abstracción para desarrollar un modelo mental adecuado para capturar la organización y arquitectura interna de las computadoras.El uso de simuladores facilita la representación visual e interactiva de procesos complejos, permitiendo que los estudiantes comprendan mejor el funcionamiento de las computadoras y ofreciendo a los docentes herramientas poderosas para conectar teoría y práctica.

En la enseñanza de la arquitectura de computadoras, los simuladores son especialmente útiles para ilustrar el funcionamiento interno de los procesadores, la ejecución de instrucciones y el manejo de la memoria. Los alumnos pueden experimentar con diferentes configuraciones y parámetros, observar el impacto en el rendimiento y comprender cómo se aplican los conceptos teóricos en la práctica. La simulación también permite explorar escenarios hipotéticos y evaluar el comportamiento de sistemas complejos sin necesidad de hardware físico. En resumen, la simulación en la educación es una herramienta valiosa para mejorar la comprensión de los alumnos, fomentar la experimentación y promover el aprendizaje activo [18][7][8].

El uso de herramientas de simulación en la enseñanza para procesos dinámicos complejos, como las operaciones intrínsecas de la computadora, que permiten representar de forma visual e interactiva la organización y arquitectura interna de la computadora, facilitando así la comprensión de su funcionamiento por parte de los alumnos y el desarrollo de los temas por parte del docente. En este contexto, los simuladores juegan una pieza clave en el campo de la Arquitectura de Computadoras, permitiendo conectar fundamentos teóricos con la experiencia práctica, implicando abstracciones y haciendo más rica la labor docente. Los simuladores, como **SimpleScalar**, **SPIM** y **GEM5**, permiten a los estudiantes experimentar con arquitecturas complejas y técnicas avanzadas como el pipelining y la ejecución fuera de orden, proporcionando una representación visual e interactiva que enriquece la comprensión teórica y práctica.

En este contexto, los simuladores juegan un papel clave en la enseñanza de la Arquitectura de Computadoras, al conectar los fundamentos teóricos con la experiencia práctica, permitiendo abstraer los conceptos y hacer más rica la labor docente.

2.10 El Formalismo DEVS (Discrete Event System Specification)

DEVS, la abreviación de Discrete Event System Specification, es un formalismo modular y jerárquico para el modelado y análisis de sistemas que pueden ser representados como sistemas de eventos discretos, sistemas de estado continuo o sistemas híbridos. Este formalismo, desarrollado por Bernard P. Zeigler en los años 70, extiende el concepto de las máquinas de Moore al proporcionar una estructura para modelar sistemas complejos mediante la utilización de eventos cronometrados [20].

2.10.1 Descripción del formalismo DEVS

El formalismo DEVS define el comportamiento de un sistema real utilizando eventos de entrada y salida, y transiciones entre estados concretos. Un sistema en DEVS está compuesto por modelos atómicos y acoplados. Los modelos atómicos representan las unidades básicas de comportamiento, mientras que los modelos acoplados consisten en combinaciones de modelos atómicos y/o otros modelos acoplados. Esta estructura jerárquica facilita la gestión y análisis de sistemas complejos, permitiendo la prueba de subsistemas de manera aislada antes de integrarlos en el sistema completo. Bajo un punto de vista general, un modelo DEVS está caracterizado por generar eventos de salida Y , en relación con el estado en el que se encuentre S y las entradas recibidas X, cada cierto tiempo.

2.10.2 Aplicaciones del formalismo DEVS

El formalismo DEVS es aplicable a una amplia gama de sistemas, desde redes de comunicación hasta procesos de manufactura. Por ejemplo, en una red de comunicación, un modelo DEVS puede simular el enrutamiento de paquetes de datos y la gestión de congestiones. En la manufactura, un modelo DEVS puede representar el flujo de materiales y el control de calidad en una línea de producción, ayudando a identificar cuellos de botella y optimizar procesos.

Capítulo 3

Comparativa de simuladores

En este capítulo, se analizan y comparan varios simuladores x86 que podrían integrarse en la asignatura Arquitectura de Computadoras de la Licenciatura en Sistemas.

La selección y evaluación de estos simuladores se ha basado en criterios específicos diseñados para medir su efectividad en un entorno educativo, con el objetivo de identificar las herramientas que mejor apoyen el proceso de enseñanza y aprendizaje. Los criterios de evaluación incluyen aspectos como usabilidad, editor de código, documentación, ejecución de simulación, nivel de especificación de la arquitectura x86, características del producto y cobertura de los contenidos curriculares.

Los resultados de esta comparativa se publicaron en el XVII Congreso de Tecnología en Educación y Educación en Tecnología en 2022, bajo el título 'Herramientas de software para dar soporte en la enseñanza y aprendizaje de la arquitectura x86' [34].

3.1 Estudios similares

Cabe destacar que existen antecedentes de estudios comparativos que evalúan diferentes simuladores aplicados a la enseñanza de los cursos de arquitectura de computadoras: - "A survey and evaluation of simulators suitable for teaching courses in computer architecture and organization", 2009 [11]: Otros estudios evalúan diferentes simuladores para abordar diferentes temas en el dictado de los cursos de Arquitectura de Computadoras, en general estos estudios evalúan simuladores en términos de dos categorías predefinidas: una referida a las características de la simulación, como ser granularidad, usabilidad, disponibilidad, presentación visual, flujo de simulación, etc., y otra sobre la cobertura de los contenidos preestablecidos en las currículas. - "Survey and evaluation of simulators suitable for teaching for computer architecture and organization Supporting undergraduate students at Sir Syed University of Engineering & Technology", 2012 [12]: Evalúa aspectos como usabilidad, disponibilidad, funda-

mentos de arquitectura informática, jerarquía de sistemas de memoria, comunicación e interfaz y diseño de sistemas de procesadores.

Este trabajo se diferencia al proponer un enfoque diferente: evaluar los simuladores x86 bajo criterios de evaluación basados en características de simulación y en contenidos específicos de la asignatura Arquitectura de Computadoras de la carrera de Licenciatura en Sistemas de la Universidad Nacional de Entre Ríos

3.2 Simuladores bajo análisis

Un simulador de arquitectura es un software que imita una situación del mundo real y, en este contexto, puede imitar el hardware de un sistema de cómputo. Estos simuladores se enfocan principalmente en representar los aspectos arquitectónicos y funcionales del hardware. El uso de simuladores permite realizar cambios, pruebas y ejecución de programas sin temor a dañar ningún componente o por falta de la computadora. [10].

Algunos programas ofrecen una representación visual e interactiva de la organización y arquitectura interna de la computadora, facilitando así la comprensión de su funcionamiento. Entre estos se encuentran los simuladores Assembly debugger (x86), Simple 8-bit Assembler Simulator, Microprocessor Simulator, Simulador de ensamblador de 16 bits y Emu8086. En este sentido, los simuladores juegan un papel clave en el campo de la Arquitectura de Computadoras, permitiendo conectar fundamentos teóricos con la experiencia práctica, simplificando abstracciones y facilitando la labor docente [11][12][13][14][35].

3.3 Criterios de evaluación

Los criterios de evaluación se han definido cuidadosamente para asegurar una valoración integral de los simuladores. Estos criterios incluyen:

- **Usabilidad**: Se refiere a la capacidad ser usado del producto.
  - **Indicadores**:
    - Facilidad de aprendizaje: tiempo requerido para familiarizarse con la herramienta.
    - Interfaz de usuario: claridad y organización.
    - Documentación y ayuda: disponibilidad y accesibilidad de tutoriales y guías.
  - **Escala**: difícil-media-fácil.

- **Editor**: Funcionalidades que ofrece el editor para escribir y depuración de código en lenguaje ensamblador.
  - **Indicadores**:
    * Capacidad de edición: funcionalidades como resaltado de sintaxis, puntos de interrupción (breakpoints).
    * Errores de sintaxis: detección y aviso de errores.
    * Almacenamiento: opciones para guardar y cargar programas.
  - **Escala**: baja-media-alta.
- **Documentación**: Disponibilidad de la documentacion para dar soporte al aprendizaje.
  - **Indicadores**:
    * Manual de usuario: disponibilidad y calidad.
    * Tutoriales: existencia y utilidad de tutoriales de aprendizaje.
    * Repertorio de instrucciones: exhaustividad y claridad en la explicación de instrucciones.
  - **Escala**: mínima-media-completa.
- **Ejecución de simulación**: Facilidad para controlar y observar el flujo de ejecución de sus programas.
  - **Indicadores**:
    * Control de la simulación: opciones para pausar, reanudar, y retroceder la ejecución.
    * Visualización: claridad en la representación del flujo de ejecución.
    * Configurabilidad: opciones para ajustar la velocidad del reloj de la CPU y otros parámetros.
  - **Escala**: baja-media-alta.
- **Nivel de especificación de la Organización y Arquitectura del sistema simulado**: Precisión con la que el simulador representa la arquitectura x86. Nivel de implementación del set de instrucciones, memoria, módulos de E/S, etc.
  - **Indicadores**:
    * Precisión de la arquitectura: fidelidad en la representación de la arquitectura x86.
    * Implementación del set de instrucciones: completitud y exactitud.
    * Soporte de módulos: inclusión y funcionalidad de memoria, módulos de E/S, etc.
  - **Escala**: mínima-media-completa.
- **Características del desarrollo del producto software**: se basa en las características propias del producto software.

  - **Indicadores**:
    * Tipo de licencia: open source o privativa.
    * Actualizaciones: frecuencia y fecha de la última versión.
    * Plataforma: si es una aplicación web o de escritorio.
  - **Escala**: mala-buena-muy buena.

- **Cobertura de los contenidos preestablecidos en la currícula**: Cobertura de los tópicos de la asignatura Arquitectura de Computadoras.
  - **Indicadores**:
    * Correspondencia con el currículum: alineación con los tópicos de la asignatura Arquitectura de Computadoras.
    * Profundidad de los temas: nivel de detalle en la cobertura de los contenidos curriculares.
  - **Escala**: baja-media-alta.

3.4 Selección de simuladores

A partir de una exhaustiva exploración en internet sobre herramientas de simulación de la arquitectura x86 utilizadas para la enseñanza, se identificaron los siguientes simuladores: Assembly debugger (x86), Simple 8-bit Assembler Simulator, Microprocessor Simulator, Simulador de ensamblador de 16 bits, Emu8086, VonSim, Orga1 y Qsim.

La selección se basa en una evaluación preliminar que considera se seleccionaron tres considerando el tiempo de evaluación de cada uno y la cantidad de criterios cumplidos por cada simulador, buscando aquellos que ofrecen un balance adecuado entre funcionalidad, usabilidad, documentación y alineación con los contenidos curriculares de la asignatura Arquitectura de Computadoras. De estos simuladores, se seleccionaron tres que, a priori, contemplaban la mayor cantidad de criterios a evaluar: Emu8086, VonSim y Simple 8-bit Assembler Simulator.

3.5 Participantes en la evaluación

La evaluación de los simuladores aplicando los criterios preestablecidos fue llevada a cabo por 3 docentes de la asignatura arquitectura de computadoras [Marcelo A. Colombani, José M. Ruiz, Amalia G. Delduca], quienes aportaron su experiencia en el uso de simuladores y su aplicabilidad en la enseñanza de arquitectura de computadoras. Además, se contó con la colaboración de 1 asesor externo [Marcelo A. Falappa], quienes ofrecieron una visión externa y validaron el proceso de evaluación y los resultados obtenidos.

3.6 Análisis comparativo

A continuación, se presenta un análisis detallado de los simuladores seleccionados basado en los criterios de evaluación definidos:

3.6.1 Simple 8-bit Assembler Simulator

- **Usabilidad**: Nivel medio. Presenta todos los componentes en una sola pantalla, lo cual puede ser abrumador para usuarios principiantes.
- **Editor**: Nivel bajo. Incluye aviso de errores de sintaxis al ensamblar, sin resaltado de sintaxis ni breakpoints. No permite guardar o cargar programas.
- **Documentación**: Nivel mínimo. Consta solo de un manual de instrucciones implementadas.
- **Ejecución de simulación**: Nivel medio. Permite configurar la velocidad del reloj de CPU y ofrece controles básicos de simulación.
- **Nivel de especificación**: Nivel mínimo. Simplifica la arquitectura x86 en un CPU de 8 bits, con memoria de 256 bytes y sin soporte para IN y OUT.
- **Desarrollo del producto**: Nivel bueno. Licencia MIT, última versión en 2015, desarrollado sobre plataforma web.
- **Cobertura de contenidos**: Nivel bajo. No implementa memoria independiente para módulos de entrada y salida, rutinas de tratamiento de interrupciones ni ciclo de instrucción.

3.6.2 VonSim

- **Usabilidad**: Nivel medio. Presenta componentes mediante solapas, lo cual puede ser abrumador para usuarios iniciales.
- **Editor**: Nivel medio. Incluye aviso de errores de sintaxis, resaltado de sintaxis y breakpoints mediante interrupción por software.
- **Documentación**: Nivel medio. Posee manual de uso y tutorial interactivo.
- **Ejecución de simulación**: Nivel medio. Permite configurar la velocidad del reloj de CPU y ofrece controles básicos de simulación.
- **Nivel de especificación**: Nivel medio. Representa una simplificación del procesador 8088 con arquitectura de 16 bits y memoria direccionable de 16 KiB.
- **Desarrollo del producto**: Nivel muy bueno. Licencia GNU Affero General Public License v3.0, última versión en 2020, desarrollado sobre plataforma web con extensa evidencia de uso académico.
- **Cobertura de contenidos**: Nivel medio. Implementa dispositivos internos y externos, pero no desarrolla contenidos visuales para ciclo de instrucción y medidas de rendimiento.

3.6.3 Emu8086

- **Usabilidad**: Nivel fácil. Presenta inicialmente el editor y permite activar componentes del simulador a medida que se cargan programas.
- **Editor**: Nivel alto. Incluye aviso de errores de sintaxis, resaltado de sintaxis y opciones de breakpoints. Permite guardar y cargar programas.
- **Documentación**: Nivel completo. Incluye manual de instrucciones, tutorial de aprendizaje y manual de uso.
- **Ejecución de simulación**: Nivel alto. Permite configurar la velocidad del reloj de CPU y ofrece controles avanzados de simulación, incluyendo "step back".
- **Nivel de especificación**: Nivel completo. Representa detalladamente la arquitectura del procesador 8086 con memoria direccionable de 1 MiB y soporte para interrupciones por software y hardware.
- **Desarrollo del producto**: Nivel bueno. Licencia privativa, última versión en 2023, desarrollado sobre plataforma de escritorio.
- **Cobertura de contenidos**: Nivel alto. Soporta todos los modos de direccionamiento y permite emular el booteo de una IBM PC desde el floppy disk, entre otros.

Cuadro 3.1: Comparativa según criterios de evaluación preestablecidos

Criterio.de.Evaluacion	Simple.8.bit.Assembler.Simulator	VonSim	Emu8086
Usabilidad	Medio	Medio	Fácil
Editor	Bajo	Medio	Alto
Documentación	Mínima	Media	Completa
Ejecución de simulación	Medio	Medio	Alta
Nivel de especificación x86	Mínima	Media	Completa
Características del producto	Buena	Muy buena	Buena
Cobertura de contenidos	Baja	Media	Alta

3.7 Resultados

La asignatura promueve el uso de simuladores para apoyar la enseñanza y el aprendizaje, permitiendo aplicar los contenidos desarrollados en máquinas reales. Emu8086 es la herramienta más adecuada para esta finalidad, ya que facilita la implementación de programas en hardware real. Sin embargo, su dependencia de MS-DOS complica su ejecución en sistemas operativos actuales, requiriendo el uso de emuladores de MS-DOS, lo que añade complejidad al proceso de enseñanza y aprendizaje.

Desde 2018, la asignatura utiliza la versión 4.08 de Emu8086. La herramienta tiene un periodo de evaluación gratuito de 14 días, después del cual se debe adquirir una

licencia. Esto es un inconveniente, ya que se busca que los estudiantes puedan acceder a las herramientas de forma libre y gratuita.

Utilizar lenguaje NASM (Netwide Assembler) garantiza soporte tanto para Linux como Windows a través de herramientas libres como GCC (GNU Compiler Collection), generando programas para la arquitectura x86 de 16, 32 y 64 bits.

Emu8086 destaca por su interfaz dinámica, que muestra componentes como la pila, flags, teclado y pantalla solo cuando son necesarios, a diferencia de otros simuladores que presentan todos sus componentes desde el inicio.

En el criterio de evaluación dos, Emu8086 se destaca por su editor, que incluye puntos de ruptura para detener la ejecución del programa en un punto específico y retroceder a la intruccion anterior. Además, permite guardar y recuperar programas directamente desde el editor.

En el criterio de evaluación tres, Emu8086 se destaca por ofrecer tres tipos de documentación: un repertorio de instrucciones con ejemplos para cada tipo de instrucción, un manual que explica las partes de la herramienta y un tutorial para aprender a programar en ensamblador.

En el criterio de evaluación cuatro, Emu8086 se destaca por ofrecer una mayor cantidad de controladores para gestionar el flujo de ejecución, como la capacidad de retroceder la ejecución de una instrucción y recargar el programa actual.

En el criterio de evaluación cinco el emu8086 se destaca del resto debido a que ofrece una mayor especificidad de la arquitectura x86, además implementa interrupciones del sistema operativo MS-DOS, a través del cual se pueden ejecutar los programas en una máquina real.

En el criterio de evaluación seis VonSim se destaca del resto debido a que es licencia libre y posee una comunidad que respalda el proyecto.

En cuanto al último criterio, ninguna de las herramientas evaluadas cubre todos los contenidos que se pretende desarrollar con la ayuda de una herramienta, quedando excluido pasos del ciclo de instrucción y medidas de rendimientos (tiempo de CPU y CPI: ciclo por instrucción).

En resumen: - Emu8086 se destaca por su alta usabilidad, documentación completa, y precisión en la simulación de la arquitectura para la enseñanza de la arquitectura x86. Sin embargo, su licencia privativa y dependencia de MS-DOS representan desafíos importantes. - VonSim ofrece una buena solución con licencia libre y una interfaz más amigable, pero su nivel de especificación y cobertura de contenidos es medio, lo cual podría limitar su efectividad en el curso. - Simple 8-bit Assembler Simulator tiene limitaciones significativas en usabilidad, documentación, y especificación, lo que lo hace menos adecuado para una enseñanza completa de la arquitectura x86.

Del análisis realizado, se concluye que, aunque todos los simuladores evaluados presentan ventajas y desventajas, ninguno de los simuladores analizados parece adecuado para la asignatura sugiere que los simuladores actuales no cumplen completamente con los requisitos de complejidad y cobertura necesarios para la enseñanza exhaustiva de la arquitectura x86. Se recomienda usar Emu8086 mientras se desarrolla un simulador que combine sus mejores características con una licencia libre, con soporte para sistemas operativos modernos y una mayor cobertura de contenidos curriculares para la asignatura..

3.7.1 Publicación

El resultado de esta comparativa fue publicado en el XVII Congreso de Tecnología en educación y Educación en Tecnología año 2022 bajo el título de "Herramientas de software para dar soporte en la enseñanza y aprendizaje de la arquitectura x86" [34].

Durante la elaboración de este análisis comparativo establecí contacto con uno de los docentes de la Universidad Nacional de la Plata (UNLP) que habían desarrollado un simulador web VonSim, con el acuerdo de docente se generó una solicitud de incorporación de cambios (pull request) en el repositorio de Github del simulador VonSim y en agosto-2023 salió una nueva versión donde se implementan animaciones de la ejecución de instrucciones y documentación on-line.

Capítulo 4

Diseño y Construcción del Simulador

En este capítulo se describe el proceso de diseño y desarrollo de una herramienta de simulación específica para la arquitectura x86, destinada a facilitar la enseñanza de los principios de arquitectura de computadoras. Se detalla tanto la justificación del diseño como los pasos seguidos para su construcción y validación.

En el capitulo anterior, se Estudió y evaluó diferentes herramientas actuales de simulación destinadas a apoyar la enseñanza de la arquitectura x86. Este objetivo se cumplió mediante una revisión exhaustiva de las herramientas de simulación disponibles y posterior publicación de los resultados.

Para cumplir con este objetivo específico, la herramienta debe satisfacer una serie de requisitos previamente establecidos. A partir de estos, se diseña la arquitectura, se desarrolla el simulador, y se realizan pruebas exhaustivas para validar su funcionalidad.

4.1 Requisitos de la Herramienta

La herramienta debe cumplir con los siguientes requisitos: 1. **Visión global de la estructura y funcionamiento de la computadora:** - Mostrar la estructura completa de la computadora (CPU, bus, memoria y E/S) durante la ejecución de programas, destacando componentes relevantes.

2. **Generación y ejecución de programas escritos en ensamblador:**
    - Permitir tanto la ejecución paso a paso como completa, facilitando la comprensión de cada instrucción.

3. **Repertorio de instrucciones x86 reducido y habilitado progresivamente:**
   - Seleccionar un subconjunto esencial de instrucciones x86, habilitándolas progresivamente según avance el contenido de la asignatura.
4. **Simulación visual e interactiva de micropasos de una instrucción:**
   - Utilizar el lenguaje de transferencia entre registros (RTL) para describir el flujo de datos, facilitando la comprensión del ciclo básico de una instrucción. Correr un programa y que se ilumine cada componente durante la ejecución de un programa.
5. **Gestión básica de interrupciones y comunicación con periféricos:**
   - Incorporar un vector de interrupción predefinido para interactuar con el teclado y monitor. Incluir instrucciones IN y OUT, y un módulo de E/S simplificado.
6. **Medidas de rendimiento sobre la ejecución de un programa:**
   - Proveer información sobre tiempo de ciclo, tiempo de CPU y CPI de un programa.

4.2 Arquitectura propuesta

La arquitectura propuesta, de 8 bits, es una versión simplificada de la x86, diseñada con un repertorio de instrucciones reducido y un conjunto de componentes fundamentales. Esta simplificación permite a los estudiantes comprender conceptos clave como el formato de instrucciones, modos de direccionamiento y el ciclo de búsqueda y ejecución. Este enfoque facilita un aprendizaje progresivo y accesible para los estudiantes. La decisión de simplificar el conjunto de instrucciones y la arquitectura del simulador desarrollado para este proyecto se fundamenta en principios pedagógicos que buscan facilitar el aprendizaje de conceptos fundamentales de la arquitectura x86. La arquitectura x86 es notoriamente compleja, con un conjunto de instrucciones extenso y diversas características avanzadas que pueden resultar abrumadoras para los estudiantes que se inician en el estudio de la arquitectura de computadoras. Al simplificar el conjunto de instrucciones y reducir la complejidad de la arquitectura en el simulador, se busca disminuir la carga cognitiva que los estudiantes deben manejar, permitiéndoles enfocarse en entender los principios básicos sin distracciones innecesarias.

La simplificación permite que el simulador se centre en los aspectos esenciales de la arquitectura x86, como el ciclo de instrucción, la interacción entre los distintos componentes de la CPU, y el flujo básico de datos dentro del sistema. Al eliminar instrucciones y características avanzadas que no son cruciales para la comprensión de

estos fundamentos, los estudiantes pueden desarrollar una comprensión sólida de los conceptos clave antes de abordar las complejidades adicionales de la arquitectura x86 completa.

El diseño del simulador también está alineado con un enfoque de aprendizaje progresivo, en el cual los estudiantes pueden comenzar con un conjunto de instrucciones reducido y una arquitectura simplificada, y luego avanzar hacia simuladores y herramientas más complejas que modelan la arquitectura x86 en su totalidad. Esta progresión gradual es pedagógicamente sólida, ya que permite a los estudiantes construir sobre conocimientos previos y asimilar conceptos más avanzados a medida que se sienten más cómodos con los fundamentos.

Un conjunto de instrucciones simplificado facilita la implementación de ejercicios y prácticas que los estudiantes pueden completar en un tiempo razonable y con un nivel de esfuerzo que es manejable. Esto es crucial para el aprendizaje práctico, ya que los estudiantes pueden experimentar con el simulador, cometer errores, y aprender de ellos sin enfrentarse a la frustración que podría derivarse de una complejidad excesiva.

Finalmente, al utilizar un simulador con una arquitectura simplificada, se fomenta el pensamiento crítico entre los estudiantes, ya que estos pueden cuestionar y analizar las diferencias entre el modelo simplificado y la arquitectura x86 real. Este contraste impulsa a los estudiantes a reflexionar sobre las razones detrás de las características más complejas de la arquitectura x86, lo que enriquece su comprensión y les permite apreciar mejor la sofisticación de los sistemas reales.

En resumen, la simplificación del conjunto de instrucciones y la arquitectura en el simulador no es un sacrificio de precisión, sino una estrategia pedagógica deliberada para mejorar la comprensión de los estudiantes. Estas decisiones de diseño están orientadas a hacer que los conceptos fundamentales sean más accesibles y comprensibles, lo cual es esencial para establecer una base sólida en la arquitectura x86 antes de abordar su complejidad total.

Consta de los siguientes componentes:

4.2.1 Componentes

Procesador o CPU cuenta con las siguientes características:

Cuadro 4.1: Arquitectura x86 propuesta

Arquitectura	Características
Arquitectura	- von Neumann, memoria de datos e instrucciones compartidas.
Registros	- 4 registros de propósito general, `AL`, `BL`, `DL` y `DL` de 8 bits.
	- 2 registros de propósito específico `IP` y `SP`.

Arquitectura	Características
	- 1 registro de estado para las banderas `RS`.
	- 2 registros Buffer entre CPU y memoria `MAR`: buffer de direcciones y `MBR`: buffer de datos.
Memoria	- contiene 256 posiciones y cada una contiene un byte.
Buses	- Bus de datos y direcciones de 8 bits.
Instrucciones	- Tamaño de instrucciones variable de 1, 2 y 3 bytes.
	- 16 instrucciones soportadas.

4.2.2 Repertorio de instrucciones

El repertorio de instrucciones x86 propuesto para facilitar el aprendizaje y la enseñanza de la arquitectura x86 para aquellos que están recién familiarizándose con este conjunto de instrucciones, es un repertorio ficticio basado en la arquitectura x86, para facilitar el aprendizaje se decidió simplificar los conceptos a un repertorio de 8 bits para un mejor entendimiento.

Primera etapa

En un primera instancia se presentaran solo algunas instrucciones necesarias para desarrollar los primeros contenidos de la asignatura:

Cuadro 4.2: Tabla de Instrucciones y Códigos de Operación de la Arquitectura x86

Instrucciones	Código operación	nemónico	Acción
Transferencia de datos	{0, 1, 2}	MOV	0- Copiar entre registros
			1- Cargar a registro
			2- Almacenar en memoria
Pocesamiento de datos	{3, 4, 5}	ADD	3-4-5 Sumar
	{6, 7, 8}	SUB	6-7-8 Restar
	{9, 10, 11}	CMP	9-10-11 Comparar
Control de flujo	{12}	JMP	12-a Salto incondicional
		JZ	12-b Salto condicional si Z=1
		JC	12-c Salto condicional si C=1

Formato de instrucciones El formato de las instrucciones propuesto para la enseñanza de la arquitectura es una simplificación de la arquitectura x86, siendo este último un set CISC (conjunto de repertorio de instrucciones complejas) las instrucciones tienen diferente tamaño para poder aplicar estos conceptos a la enseñanza se

representa un repertorio de instrucciones simplificado. Primero veremos la codificaciones de 4 tipos de instrucciones.

Cuadro 4.3: Tabla de codififación de instrucciones

Casos	Codificación	Parámetros
A: entre registros	—- XXYY	XX = Registro destino, YY = Registro fuente
B: Cargar a registro	—- XX00 DDDDDDDD	XX = Registro destino, D = Dirección de memoria
C: Almacenar en memoria	—- 00YY DDDDDDDD	YY = Registro fuente, D = Dirección de memoria
D: control de flujo	—- ffff DDDDDDDD	ffff = funcionalidad, D = Dirección de memoria

Considerando:

- `A`: operaciones entre registros del procesador.
- `B`: operaciones que cargan en registros del procesado.
- `C`: operaciones que almacenan en memoria.
- `D`: operaciones de control de flujo del programa.
- `----`: Código de operación de la instrucción.
- `XX` o `YY`: Índices de registros, número entre `0` y `3`.
- `D`: Dirección de memoria, número de 8 bits.
- `ffff`: representan el comportamiento de la instrucción.
- Las instruciones de transferencia y de procesamiento soportan los formatos `A`, `B` y `C`.
- Las instruciones de control de flujo soportan el formato `D`.

Registros (Banco de Registros)

Número de registros (R)	Binario	Decimal
AL	00	0
BL	01	1

Número de registros (R)	Binario	Decimal
CL	10	2
DL	11	3

Modos de direccionamientos Los siguientes tipos de direccionamiento son utilizados en las instrucciones de esta CPU para referenciar a los operandos involucrados en la instrucción:

Operando	tipo
`Rx`,`Ry`	Registro a registro: los operandos de la instrucción son registros
`[M]`	Directo: un operando de la instruccion es el contenido de una dirección de memoria, se accede a un contenido de memoria
`M`	Inmediato: el operando fuente de la instrucción es un valor contenido en la misma instrucción

Se puede implementar las siguientes instrucciones con estos modos de direccionamiento y formato de instrucciones:

#	Instrucción	Acción	Codificación
0	MOV Rx, Ry	Rx ← Ry	0000 XXYY
1	MOV Rx, [M]	Rx ← Mem[Dirección]	0001 00YY DDDDDDDD
2	MOV [M], Ry	Mem[Dirección] ← Rx	0010 XX00 DDDDDDDD
3	ADD Rx, Ry	Rx ← Rx + Ry	0011 XXYY
4	ADD Rx, [M]	Rx ← Rx + Mem[Dirección]	0100 00YY DDDDDDDD
5	ADD [M], Ry	Mem[Dirección] ← Mem[Dirección] + Ry	0101 XX00 DDDDDDDD
6	SUB Rx, Ry	Rx ← Rx - Ry	0110 XXYY
7	SUB Rx, [M]	Rx ← Rx - Mem[Dirección]	0111 00YY DDDDDDDD
8	SUB [M], Ry	Mem[Dirección] ← Mem[Dirección] - Ry	1000 XX00 DDDDDDDD
9	CMP Rx, Ry	Modifica flags de Rx - Ry	1001 XXYY
A	CMP Rx, [M]	Modifica flags de Rx - Mem[Dirección]	1010 00YY DDDDDDDD
B	CMP [M], Ry	Modifica flags de Mem[Dirección] - Ry	1011 XX00 DDDDDDDD
C	JMP M	IP ← Dirección	1100 0000 DDDDDDDD
C	JC M	Si flag C=1 entonces IP ← Dirección	1100 0001 DDDDDDDD
C	JZ M	Si flag Z=1 entonces IP ← Dirección	1100 0011 DDDDDDDD

4.2.3 Segunda etapa

En un segunda instancia se amplian los modos de direccionamiento de las instrucciones y se incorporan nuevos formato de instrucciones y modos de direccionamiento.

Modos de direccionamientos

Se incoporan los siguientes modos de direccionamiento:

- Indirecto: dirección de memoria donde está el operando viene determinada por el contenido del registro `BL`

Se incorpora los siguientes operandos a las instrucciones: * **MOV *Rx, [BL]***

- **MOV *Rx, d***
- **MOV *Rx, [BL + d]***
- **MOV *[BL], Ry***
- **MOV *[BL + d], Ry***
- **MOV *[mem], [BL]***
- **MOV *[mem], d***
- **MOV *[mem], [BL + d]***

Operando	tipo
`[BL]`	indirecto por registro, Contenido de la dirección de memoria esta dada por el registro `BL`
`D` o `d`	el operando fuente es un dato inmediato
`[BL + d]`	registro `BL` + dato inmediato

Formato de instrucciones

Las instrucciones están codificadas con 1, 2 o 3 bytes. Los primeros 4 bits identifican el `opcode` de la instrucción y determinan como se tienen que ise interpretar los 4 bits restantes.

Caso	Codificación	Parámetros
A: entre registros	`---- XXYY`	`XX` = Registro destino, `YY` = Registro fuente
B: Cargar a registro	`---- XXmm DDDDDDDD`	`mm` = modo, `XX` = Registro destino, `D` = Dirección de memoria o Dato Inmediato
C: Almacenar en memoria	`---- mmmm DDDDDDDD dddddddd`	`mmmm` = modo ampliado, `D` = Dirección de memoria, `d` = Dato Inmediato
D: control de flujo	`---- ffff DDDDDDDD`	`ffff` = funcionalidad `D` = Dirección de memoria

`d` = Dato Inmediato, no puede ser destino de la instrucción.

Modo direccionamientos B: Cargar a registro		
`mm`= Modo	**`Byte`= tamaño**	**Interpretación**
00	2	directo `D` = Dirección de memoria
01	1	indirecto utiliza como operando implicito el registro `BL` y no requiere operando `D`
10	2	Inmediato `D` = Dato Inmediato
11	2	Indirecto la dirección se calcula operando implicito `BL` + Dato Inmediato

Modo direccionamientos C: Almacenar en memoria		
mmmm= Modo	**Byte= tamaño**	**Interpretación**
00YY	2	directo D = Dirección de memoria, YY = Registro fuente
01YY	1	indirecto BL, YY = Registro fuente
01YY	2	Indirecto la dirección se calcula operando implicito BL + Dato Inmediato
1100	3	Inmediato a memoria
1101	2	Inmediato a memoria mediante indirecto BL
1110	3	Inmediato a memoria mediante indirecto BL+ Dato Inmediato

#	Instrucción	Acción	Codificación
0	MOV Rx, Ry	Rx ← Ry	0000 XXYY
1	MOV Rx, [M]	Rx ← Mem[Dirección]	0001 XX00 DDDDDDDD
1	MOV Rx, [BL]	Rx ← Mem[BL]	0001 XX01
1	MOV Rx, D	Rx ← Dato	0001 XX10 DDDDDDDD
1	MOV Rx, [BL + D]	Rx ← Mem[BL + Dato]	0001 XX11 DDDDDDDD
2	MOV [M], Ry	Mem[Dirección] ← Rx	0010 00YY DDDDDDDD
2	MOV [BL], Ry	Mem[BL] ← Rx	0010 01YY
2	MOV [BL + D], Ry	Mem[BL + Dato] ← Rx	0010 10YY DDDDDDDD
2	MOV [M], D	Mem[Dirección] ← Dato	0010 1100 DDDDDDDD dd
2	MOV [BL], D	Mem[BL] ← Dato	0010 1101 DDDDDDDD
2	MOV [BL + D], D	Mem[BL + Dato] ← Dato	0010 1110 DDDDDDDD
3	ADD Rx, Ry	Rx ← Rx + Ry	0011 XXYY
4	ADD --, --	Mismo direccionamientos que MOV	0100 ---- --------
5	ADD --, --	Mismo direccionamientos que MOV	0101 ---- -------- --
6	SUB Rx, Ry	Rx ← Rx - Ry	0110 XXYY
7	SUB --, --	Mismo direccionamientos que MOV	0111 ---- --------
8	SUB --, --	Mismo direccionamientos que MOV	1000 ---- -------- --
9	CMP Rx, Ry	Modifica *flags* de Rx - Ry	1001 XXYY
A	CMP --, --	Mismo direccionamientos que MOV	1010 ---- --------
B	CMP --, --	Mismo direccionamientos que MOV	1011 ---- -------- --
C	JMP M	IP ← Dirección	1100 0000 DDDDDDDD
C	JC M	Si flag C=1 entonces IP ← Dirección	1100 0001 DDDDDDDD

#	Instrucción	Acción	Codificación
C	`JZ M`	Si `flag Z=1` entonces `IP` ← `Dirección`	`1100 0011 DDDDDDDD`
C	`Jxx M`	Se pueden implememementar más flags y CALL	`1100 ffff --------`

4.2.4 Tercera etapa

Se incorporan las instrucciones de manejo de pila y de manejo de interrupciones.

Cuadro 4.12: Tabla de Instrucciones y Códigos de Operación de la Arquitectura x86

Código operación	Instrucciones	mnémonico	Acción
{0, 1, 2}	Transferencia de datos	MOV	1- Copiar entre registros 2- Cargar a registro 3- Almacenar en memoria
{3, 4, 5} {6, 7, 8} {9, 10, 11}	Procesamiento de datos	ADD, SUB, CMP	1- Sumar 2- Restar 3- Comparar
{12}	Control de flujo	JMP, Jxx,CALL, RET	Salto incondicional JMP. Saltos condicionales Jxx.Llamadas a rutinas CALL y retorno RET
{13}	Manejo de pila y E/S	PUSH, POP, OUT, IN	Poner en la pila PUSH Retirar de la pila POP Enviar un byte al puerto del dispositivo de E/S Recibir un byte del dispositivo de E/S
{14}	Manejo de interrupción	INT , IRET	Llamar a una rutina de tratamiento de interrupción INT Retornar de una rutina de tratamiento de interrupción IRET
{15}	Control del CPU	NOP , HLT	No opera NOP Detiene el CPU HLT

4.2.5 Ciclo de la instrucción (Etapas de captación y ejecución)

Etapa de Captación (*):

```
1.  MAR $\leftarrow$ IP
2.  MDR $\leftarrow$ read(Memoria[MAR]); IP $\leftarrow$ IP + 1
3.  IR  $\leftarrow$ MDR
```

(*): Todas las instrucciones tienen los mismos pasos en la etapa de captación.

Etapa de Ejecución:

- 0: Copiar entre registros: `MOV Rx, Ry`
  1. Rx ← Ry
- 1: Cargar a registro:
  a) Directo: MOV Rx, [Dirección]
     - MAR ← IP
     - MBR ← read(Memoria[MAR]); IP ← IP + 1 (Obtener byte 2)

     - MAR ← MBR
     - MBR ← read(Memoria[MAR]) (Obtener operando)
     - Rx ← MBR
- 2: Almacenar en memoria:
  a) MOV [Dirección], Ry
     - MAR ← IP
     - MBR ← read(Memoria[MAR]); IP ← IP + 1 (Obtener byte 2)
     - MAR ← MBR
     - MBR ← Ry
     - write(Memoria[MAR]) ← MBR

4.3 Pruebas

Después de diseñar la herramienta, se debe construir y probar. Se deben realizar pruebas exhaustivas para asegurarse de que la herramienta cumpla con todos los requisitos y sea fácil de usar. Si se encuentran problemas durante las pruebas, se deben solucionar antes de lanzar la herramienta.

- **Pruebas Exhaustivas:** Realizar pruebas rigurosas para asegurar que la herramienta cumple con todos los requisitos. Las pruebas incluirán:
  - Validación de la funcionalidad completa de la herramienta.
  - Evaluación de la usabilidad por parte de estudiantes y profesores.
  - Identificación y solución de problemas antes del lanzamiento.

4.4 Portabilidad

Para asegurar la portabilidad, el simulador se implementará como una aplicación web que puede ejecutarse en cualquier navegador web, garantizando su uso multiplataforma.

4.5 Mantenibilidad

El código de la herramienta será modular y bien documentado, facilitando futuras actualizaciones y mantenimiento. Además, se implementarán prácticas de desarrollo sostenibles para asegurar su longevidad.

4.6 Escalabilidad

Desde la asignatura se incentiva el uso de simuladores para dar apoyo a los proceso de enseñanza y aprendizaje, pero también se incentiva que los contenidos desarrollados puedan volcarse en máquinas reales, en este sentido consideramos que el enfoque planteado por la herramienta emu8086 es el más adecuado para la asignatura, ya que facilita mecanismos para implementar los programas en máquinas reales. Sin embargo presenta el inconveniente que genera ejecutables dependientes del sistema operativo MS-DOS, la mayoría de los sistemas operativos actuales no permiten la ejecución de dichos programas, obligando a la utilización de emuladores de MS-DOS para poder correrlos, siendo esto otro elemento más que se incorpora a los procesos de enseñanza y aprendizaje. Utilizar lenguaje NASM (Netwide Assembler) garantiza soporte tanto para Linux como Windows a través de herramientas libres como GCC (GNU Compiler Collection), generando programas para la arquitectura x86 de 16, 32 y 64 bits.

Bibliografía

[1] M. A. Colombani, M. A. Falappa, A. G. Delduca, and J. M. Ruiz, "PID novel 7065: Enseñanza/aprendizaje de asignatura Arquitectura de Computadoras con herramientas de simulación de sistemas de cómputos." Feb. 2022. Accessed: Jul. 10, 2024. Available: https://proyectos.uner.edu.ar/aplicacion.php?ah=st668e6d47663eb&ai=gestion_extinv%7C%7C23000105

[2] J. Banks, J. S. Carson, B. L. Nelson, and D. M. Nicol, *Discrete-event system simulation*, 5th ed. Prentice Hall, 2010.

[3] A. M. Law, *Simulation Modeling & Analysis*, 5th ed. New York, NY, USA: McGraw-Hill, 2015.

[4] S. Robinson, *Simulation: The Practice of Model Development and Use*, 2nd edition. 2014.

[5] C. Lion, "Los simuladores. Su potencial para la enseñanza universitaria," *Cuadernos de Investigación Educativa*, vol. 2, no. 12, pp. 53–66, 2005.

[6] G. Contreras, R. G. Torres, and M. S. R. Montoya, "Uso de simuladores como recurso digital para la transferencia de conocimiento," *Apertura: Revista de Innovación Educativa*, vol. 2, no. 1, pp. 86–100, 2010.

[7] A. Garcia-Garcia, J. C. Saez, J. L. Risco-Martin, and M. Prieto-Matias, "PB-BCache: An open-source parallel simulator for rapid prototyping and evaluation of cache-partitioning and cache-clustering policies," *Journal of Computational Science*, vol. 42, p. 101102, 2020.

[8] B. Nova, J. C. Ferreira, and A. Araújo, "Tool to support computer architecture teaching and learning," in *Engineering Education (CISPEE), 2013 1st International Conference of the Portuguese Society for*, IEEE, 2013, pp. 1–8.

[9] B. Mustafa, "Evaluating A System Simulator For Computer Architecture Teaching And Learning Support," *Innovation in Teaching and Learning in Information and Computer Sciences*, vol. 9, no. 1, pp. 100–104, 2010, doi: 10.11120/ital.2010.09010100.

[10] Z. Radivojevic, M. Cvetanovic, and J. Đordevic, "Design of the simulator for teaching computer architecture and organization," in *2011 Second Eastern European*

Regional Conference on the Engineering of Computer Based Systems, IEEE, 2011, pp. 124–130.

[11] B. Nikolic, Z. Radivojevic, J. Djordjevic, and V. Milutinovic, "A Survey and Evaluation of Simulators Suitable for Teaching Courses in Computer Architecture and Organization," *IEEE Transactions on Education*, vol. 52, no. 4, pp. 449–458, Nov. 2009, doi: 10.1109/TE.2008.930097.

[12] R. Hasan and S. Mahmood, "Survey and evaluation of simulators suitable for teaching for computer architecture and organization Supporting undergraduate students at Sir Syed University of Engineering & Technology," in *Control (CONTROL), 2012 UKACC International Conference on*, IEEE, 2012, pp. 1043–1045.

[13] J. L. Hennessy and D. A. Patterson, *Computer architecture: A quantitative approach*, Fifth Edition. Elsevier, 2012.

[14] W. Stallings, *Computer organization and architecture: Designing for performance*, Eleventh Edition. Pearson, 2013.

[15] Intel, "64 and IA-32 architectures software developers manual," *325462-060US*, vols. 1, 2A, 2B, 2C, 2D, 3A, 3B, 3C and 3D, p. 4670, 2016, Available: https://software.intel.com/en-us/articles/intel-sdm

[16] AMD, "Developer Guides, Manuals & ISA Documents." Apr. 2019. Accessed: Apr. 21, 2019. Available: https://developer.amd.com/resources/developer-guides-manuals/

[17] P. Abel, *IBM PC Assembly Language and Programming*, Fifth Edition. Upper Saddle River, NJ, USA: Prentice Hall PTR, 2000.

[18] D. Skrien, "CPU Sim 3.1: A tool for simulating computer architectures for computer organization classes," *Journal on Educational Resources in Computing (JERIC)*, 2001.

[19] J. L. Peterson, *Petri net theory and the modeling of systems*. Prentice Hall PTR, 1981.

[20] B. Zeigler, H. Prähofer, and T. G. Kim, "Theory of Modeling and Simulation: Integrating Discrete Event and Continuous Complex Dynamic Systems," vol. 2, Jan. 2000.

[21] B. P. Zeigler, A. Muzy, and E. Kofman, *Theory of Modeling and Simulation: Discrete Event & Iterative System Computational Foundations*. Academic Press, 2018.

[22] L. Null and J. Lobur, *Essentials of computer organization and architecture*. Jones & Bartlett Publishers, 2014.

[23] Patterson and others, *Computer organization and design: The hardware/software interface-5th*. Morgan Kaufmann, 2014.

[24] A. S. Tanenbaum, *Structured computer organization*. Pearson Education, 2013.

[25] M. J. Murdocca and V. Heuring, *Principles of computer architecture*. Pearson Education, 2000.

[26] A. Waterman, Y. Lee, D. A. Patterson, and K. Asanovic, "The risc-v instruction set manual, volume i: User-level isa," *CS Division, EECE Department, University of California, Berkeley*, p. 28, 2014.

[27] K. R. Irvine and L. B. Das, *Assembly language for x86 processors*. Prentice Hall, 2011.

[28] B. International, *Turbo assembler user's guide*. Borland International, 1993.

[29] M. Corporation, *Microsoft macro assembler 6.1 reference*. Microsoft Press, 1992.

[30] The NASM Project, *The netwide assembler (nasm) manual*. 2023.

[31] R. Hyde, *The art of assembly language*. No Starch Press, 2010.

[32] A. Stork, C.-A. Thole, S. Klimenko, I. Nikitin, L. Nikitina, and Y. Astakhov, "Towards interactive simulation in automotive design," *The Visual Computer*, vol. 24, pp. 947–953, 2008.

[33] F. Jentsch and M. Curtis, *Simulation in aviation training*. Routledge, 2017.

[34] M. A. Colombani, J. M. Ruiz, A. G. Delduca, and M. A. Falappa, "Herramientas de software para dar soporte en la enseñanza y aprendizaje de la arquitectura x86," 2022. Accessed: Jul. 10, 2024. Available: http://sedici.unlp.edu.ar/handle/10915/139908

[35] P. BEHROOZ, *Computer Architecture: From Microprocessors to Supercomputers*. Oxford University Press Inc, 2005.